Blynt — Master 100 Use Cases: How the System Works

Complete Step-by-Step Operational Journeys & Visual Flowcharts Across All 100 Scenarios

Document Purpose: This document provides an exhaustive, step-by-step explanation of **how the system works** for all 100 use cases across the Blynt cloud accounting platform. Each use case details the interaction between the user, the frontend interface, the backend business engine, and the PostgreSQL database, accompanied by an individual architectural flowchart and explicit failure guardrails.

UC-001: Standard Email & Password Login

Module: Authentication & Identity | **Actor:** Any Registered User | **Rejection:** 401 INVALID_CREDENTIALS

Business Goal: Log into the system securely and load the user's workspace.

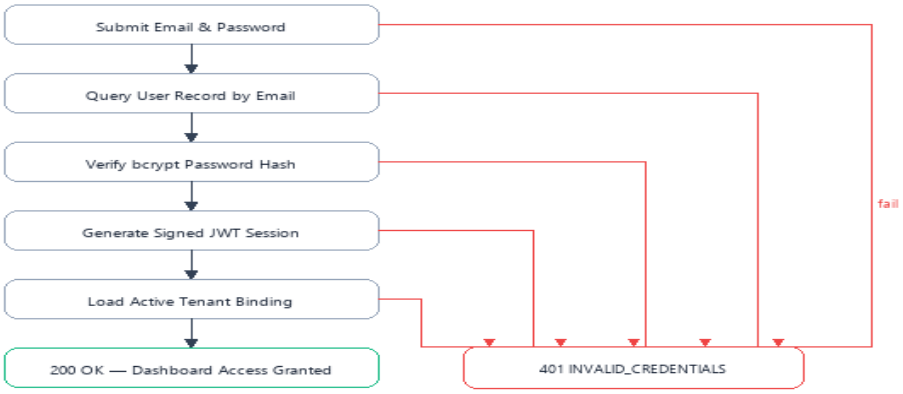


Figure 001 — Standard Email & Password Login Flow Architecture

How the System Works: 1. User enters registered email and password on the Blynt web login screen. ➔ 2. The browser sends an encrypted POST request over HTTPS to the NestJS /auth/login endpoint. ➔ 3. Blynt queries PostgreSQL for the user's record and extracts the salted bcrypt password hash. ➔ 4. The password engine compares the input against the stored hash using constant-time evaluation. ➔ 5. Upon a match, the system signs a JWT access token (15-min expiry) and sets an httpOnly refresh cookie.
Outcome: The user lands on their active organization dashboard with their role and permissions loaded. | **Guardrail:** If the password does not match, a friendly 'Invalid email or password' message appears and a failed attempt is logged.

UC-002: Multi-Tenant Workspace Context Switching

Module: Multi-Tenancy | **Actor:** Business Owner / Multi-Org Director | **Rejection:** 403 TENANT_MEMBERSHIP_REQUIRED

Business Goal: Switch between multiple businesses without logging out.



Figure 002 — Multi-Tenant Workspace Context Switching Flow Architecture

How the System Works: 1. User clicks the organization selector dropdown in the top navigation bar. ➔ 2. The UI displays all companies where the user holds an active membership. ➔ 3. User selects another company (e.g., 'Acme Retail Ltd'). ➔ 4. The backend verifies that the user's ID has an active membership record in the target company. ➔ 5. Blynt issues a new scoped tenant token, swaps the active database tenant context, and refreshes the view.

Outcome: The workspace switches completely to Acme Retail Ltd; no data from the previous company remains on screen. | **Guardrail:** If a user attempts to tamper with the URL to view an unauthorized company, access is blocked immediately.

UC-003: Session Token Expiry & Silent Refresh

Module: Session Management | **Actor:** Active User Working in System | **Rejection:** 401 REFRESH_TOKEN_EXPIRED

Business Goal: Keep the user logged in seamlessly while maintaining high security.

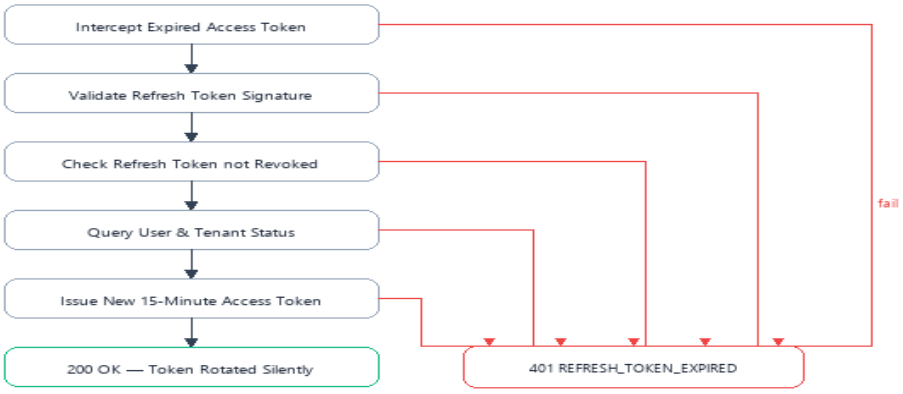


Figure 003 — Session Token Expiry & Silent Refresh Flow Architecture

How the System Works: 1. The short-lived 15-minute access token expires while the user is working. ➔ 2. On the next API request, the browser receives a token expiration signal. ➔ 3. The frontend automatically issues a silent refresh request using the secure httpOnly cookie. ➔ 4. The auth service verifies the refresh token's cryptographic signature and checks that it has not been revoked. ➔ 5. A new 15-minute access token is generated and the user's work continues with zero interruption.
Outcome: The user never gets kicked out mid-task as long as they remain active. | **Guardrail:** If the refresh token itself has expired (e.g., after 7 days of inactivity), the user is gracefully redirected to the login screen.

UC-004: Cross-Tenant Injection Prevention Guard

Module: Multi-Tenant Security | **Actor:** Platform Security Interceptor | **Rejection:** 403 TENANT_MISMATCH

Business Goal: Prevent any tenant from accidentally or maliciously accessing another tenant's financial data.

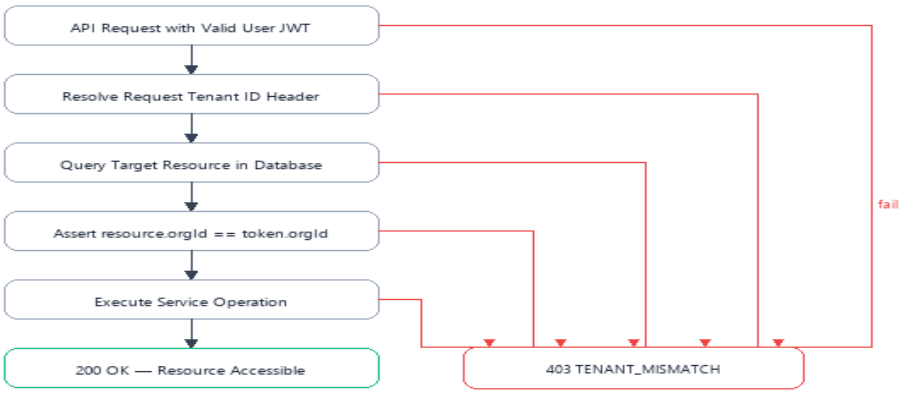


Figure 004 — Cross-Tenant Injection Prevention Guard Flow Architecture

How the System Works: 1. A user sends a request to view or edit an invoice, passing an invoice ID. ➔ 2. Before executing any query, Blynt's TenantGuard extracts the organization ID from the verified user token. ➔ 3. The database query enforces a strict WHERE filter: 'WHERE id = :invoiceId AND organization_id = :tokenOrgId'. ➔ 4. If the invoice belongs to a different company, the query returns zero rows. ➔ 5. The system intercepts the attempt, logs a potential security incident, and halts execution.
Outcome: Users can only ever see and touch transactions belonging to their own organization. | **Guardrail:** An attempt to view another business's data is rejected with a strict security access denial.

UC-005: Two-Factor Authentication (2FA) TOTP Verification

Module: Account Security | **Actor:** Security-Conscious User | **Rejection:** 401 INVALID_TOTP_CODE

Business Goal: Protect sensitive company finances with an extra mobile authentication step.

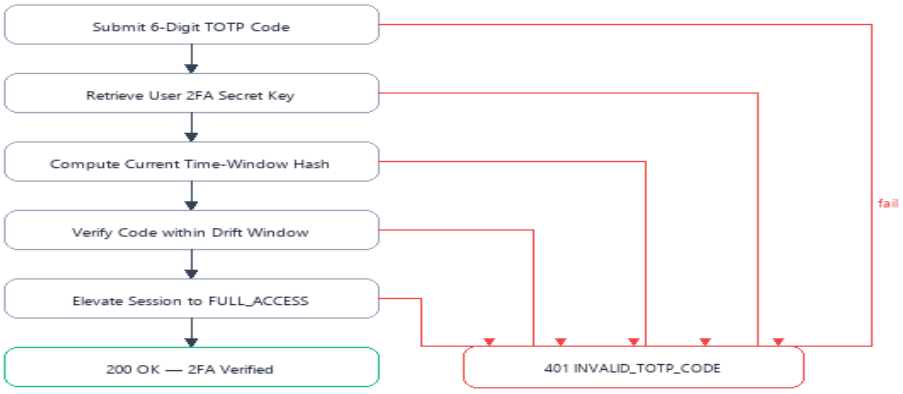


Figure 005 — Two-Factor Authentication (2FA) TOTP Verification Flow Architecture

How the System Works: 1. User enters their correct email and password. ➔ 2. The system detects that Two-Factor Authentication is enabled on the account. ➔ 3. The screen prompts the user to enter the 6-digit code from their mobile authenticator app (Google/Microsoft). ➔ 4. The backend calculates the expected HMAC-SHA1 code for the current 30-second time window. ➔ 5. If the codes match, the session is elevated to fully authenticated status and the dashboard opens.
Outcome: Account is protected against stolen passwords; only the person holding the physical phone can log in. | **Guardrail:** Entering the wrong 6-digit code or letting it expire blocks entry with a clear 'Invalid authentication code' message.

UC-006: Password Reset via Time-Limited Token

Module: Account Recovery | **Actor:** User Locked Out of Account | **Rejection:** 400 EXPIRED_RESET_TOKEN

Business Goal: Regain access to the account without administrator intervention.

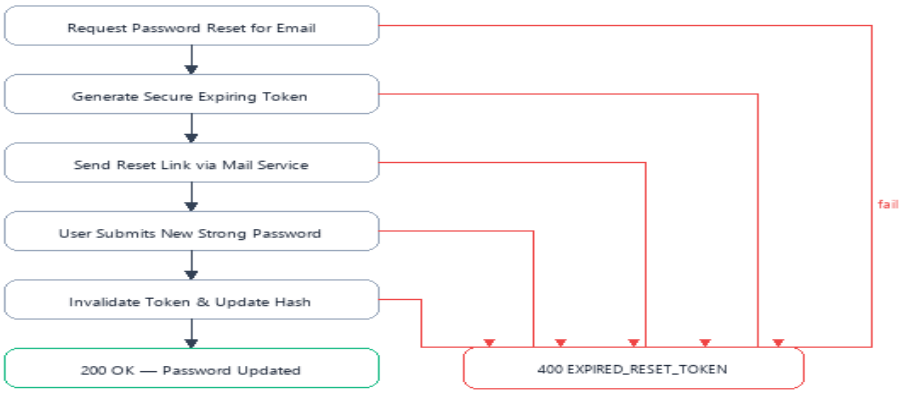


Figure 006 — Password Reset via Time-Limited Token Flow Architecture

How the System Works: 1. User clicks 'Forgot Password' and enters their registered email address. ➔ 2. Blynt generates a cryptographically random, one-time token that expires in exactly 60 minutes. ➔ 3. An automated email is sent to the user containing a secure password reset link. ➔ 4. User clicks the link and types a new, strong password matching security policies. ➔ 5. Blynt updates the salted hash in the database and immediately invalidates the reset token.
Outcome: User logs in with their new password; old passwords no longer work. | **Guardrail:** Clicking an expired or already-used reset link displays 'This reset link has expired. Please request a new one.'

UC-007: Brute-Force Login Rate Limiting Defense

Module: Cybersecurity | **Actor:** Automated Security Shield | **Rejection:** 429 TOO_MANY_REQUESTS

Business Goal: Prevent automated bot attacks from guessing user passwords.

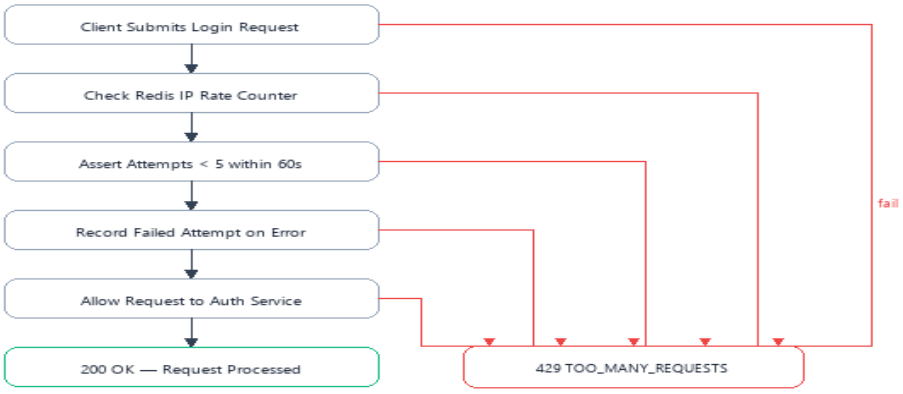


Figure 007 — Brute-Force Login Rate Limiting Defense Flow Architecture

How the System Works: 1. A malicious bot or unauthorized user rapidly submits incorrect login attempts. ➔ 2. Blynt's rate-limiting middleware tracks the number of failed attempts per IP address in Redis memory. ➔ 3. On the 5th consecutive failed attempt within 60 seconds, the rate-limiter triggers an automatic lock. ➔ 4. Further login requests from that IP address are temporarily blocked for 15 minutes. ➔ 5. The real account owner is unaffected on other devices, and a security alert is recorded in the platform log.
Outcome: The system prevents automated password dictionary attacks without crashing the server. | **Guardrail:** The screen shows: 'Too many failed login attempts. Please wait 15 minutes before trying again.'

UC-008: Platform Super Admin Portal Gate

Module: System Governance | **Actor:** Blynt Support & DevOps Team | **Rejection:** 403 SUPER_ADMIN_REQUIRED

Business Goal: Manage platform health and tenant subscriptions safely.

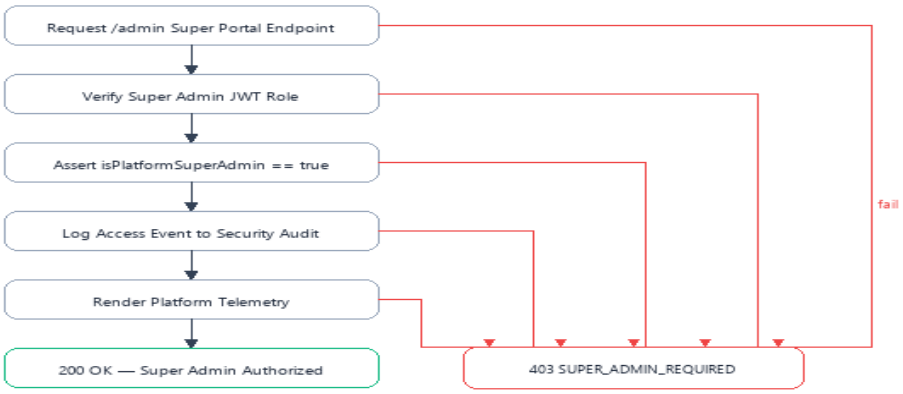


Figure 008 — Platform Super Admin Portal Gate Flow Architecture

How the System Works: 1. A user attempts to navigate to the restricted '/admin' route. ➔ 2. The SuperAdminGuard intercepts the incoming request before loading any administrative components. ➔ 3. The guard verifies that the user's role in the database is explicitly set to 'PLATFORM_SUPER_ADMIN'. ➔ 4. If confirmed, the platform administrative dashboard displays tenant counts, database health, and queue metrics. ➔ 5. All actions performed inside the portal are logged to a dedicated immutable super-admin audit trail.
Outcome: Platform operators can manage tenant subscriptions and system health safely. | **Guardrail:** Regular business owners and staff attempting to access '/admin' are blocked with an immediate 403 Forbidden.

UC-009: User Logout & Complete Session Revocation

Module: Account Security | **Actor:** User Leaving Computer | **Rejection:** 200 LOGOUT_SUCCESS

Business Goal: Ensure that logging out immediately terminates all access on shared devices.



Figure 009 — User Logout & Complete Session Revocation Flow Architecture

How the System Works: 1. User clicks 'Sign Out' in the user menu. ➔ 2. The web app sends a logout request to the server and wipes all access tokens from browser memory. ➔ 3. The backend immediately marks the refresh token as revoked in PostgreSQL. ➔ 4. The unique session identifier (JTI) is placed on a token blacklist to reject any residual requests. ➔ 5. The browser clears all auth cookies and redirects the user to the public homepage.
Outcome: The session is terminated; pressing the browser's 'Back' button cannot view private financial records. | **Guardrail:** Attempting to use an old token returns an unauthorized response.

UC-010: Employee Organization Invitation & Onboarding

Module: Team Management | **Actor:** Business Owner / Office Admin | **Rejection:** 409 USER_ALREADY_MEMBER

Business Goal: Invite a new team member to collaborate with a specific role.

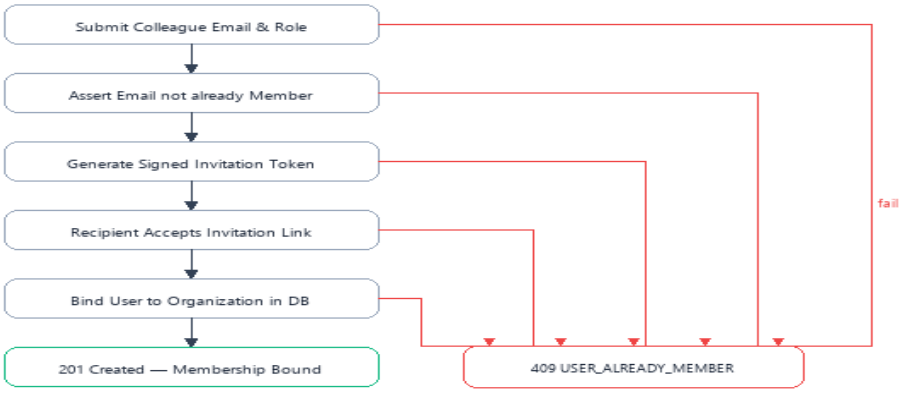


Figure 010 — Employee Organization Invitation & Onboarding Flow Architecture

How the System Works: 1. Owner enters the colleague's work email and selects their role (e.g., 'Bookkeeper'). ➔ 2. The system checks that the user is not already a member of this organization. ➔ 3. A unique invitation token is generated and emailed to the invited employee. ➔ 4. The employee clicks the link, enters their name and creates a password (or logs into an existing Blynt account). ➔ 5. The system creates the membership record in the organization with the exact role designated by the owner.
Outcome: The employee joins the organization and can immediately begin bookkeeping tasks appropriate to their role. | **Guardrail:** Attempting to invite an existing member shows 'This user is already a member of your organization.'

UC-011: Organization Legal Entity Creation

Module: Company Setup | **Actor:** New Business Owner | **Rejection:** 400 INVALID_ORG_PAYLOAD

Business Goal: Register a new legal business entity ready for UK trading.

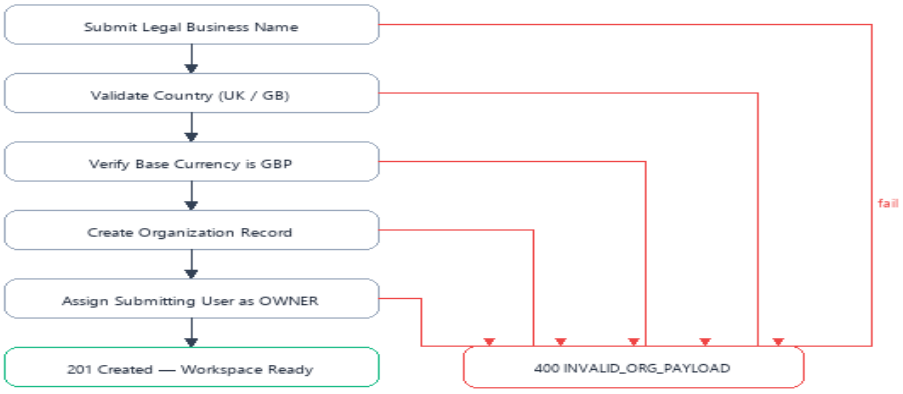


Figure 011 — Organization Legal Entity Creation Flow Architecture

How the System Works: 1. Owner completes the onboarding wizard with their legal company name (e.g., 'North Star Logistics Ltd'). ➔ 2. The system verifies that the company name is between 2 and 100 characters long. ➔ 3. Blynt sets the country jurisdiction to United Kingdom and configures the base trading currency to GBP (£). ➔ 4. The database creates the organization record and assigns the creator as the primary 'OWNER'. ➔ 5. Blynt automatically seeds standard UK Chart of Accounts nominal codes (Bank, Sales, Expenses, VAT).
Outcome: A fully configured accounting workspace is provisioned in under 2 seconds. | **Guardrail:** Leaving the company name blank displays a red warning: 'Please enter your legal company name.'

UC-012: Unique Workspace URL Slug Allocation

Module: Company Setup | **Actor:** Business Owner | **Rejection:** 409 SLUG_ALREADY_EXISTS

Business Goal: Create a clean, human-readable web address for the company workspace.

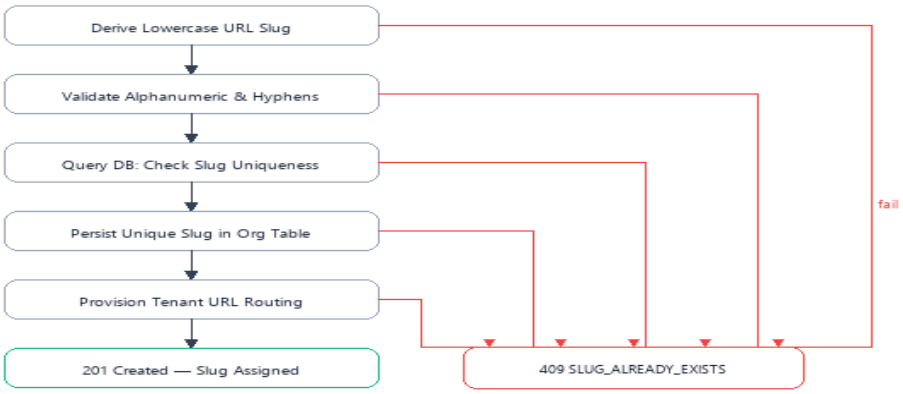


Figure 012 — Unique Organization URL Slug Allocation Flow Architecture

How the System Works: 1. As the owner types their business name, Blynt automatically generates a URL slug (e.g., 'north-star-logistics'). ➔ 2. The frontend checks that the slug only contains lowercase letters, numbers, and hyphens. ➔ 3. The backend queries the database to ensure no other company on the platform shares this slug. ➔ 4. If unique, the slug is reserved and assigned to the organization. ➔ 5. All future company URLs will cleanly use this slug (e.g., 'app.blynt.co.uk/north-star-logistics/invoices').
Outcome: The company has a clean, professional web workspace link. | **Guardrail:** If the slug is already taken by another business, Blynt suggests adding numbers (e.g., 'north-star-logistics-1').

UC-013: HMRC UK VAT Registration Number Validation

Module: Tax Configuration | **Actor:** Accountant / Business Owner | **Rejection:** 400 INVALID_VAT_FORMAT

Business Goal: Configure UK VAT so invoices and returns comply with HMRC rules.

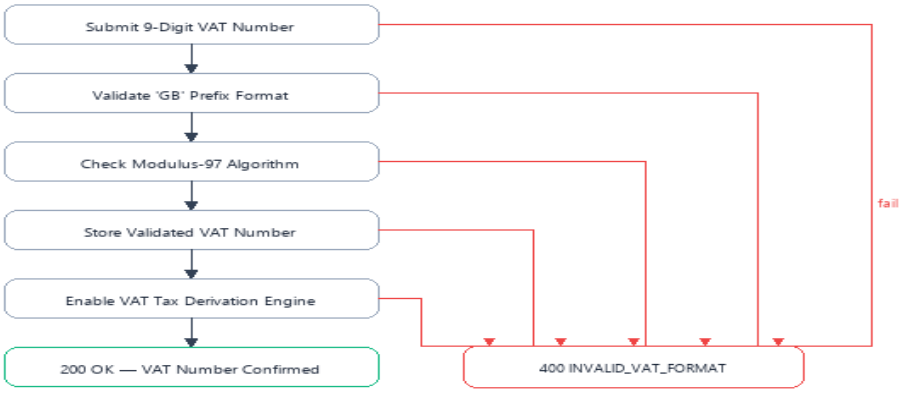


Figure 013 — HMRC UK VAT Registration Number Validation Flow Architecture

How the System Works: 1. User navigates to Settings -> Tax & VAT and enters their UK VAT registration number. ➔ 2. The system strips spaces and validates the UK format: exactly 9 digits prefixed with 'GB' (or 9 digits). ➔ 3. Blynt runs the official HMRC Modulus-97 mathematical checksum algorithm on the digits. ➔ 4. If the checksum passes, the VAT number is stored and displayed on all outbound customer invoices. ➔ 5. The platform activates standard UK VAT rates: 20% Standard, 5% Reduced, and 0% Zero-Rated.
Outcome: Invoices automatically print the valid UK VAT number, ensuring full tax deductibility for corporate clients. | **Guardrail:** Typing a fake or mistyped number displays: 'Invalid UK VAT registration number format.'

UC-014: UK Companies House Registration Validation

Module: Statutory Configuration | **Actor:** Business Owner | **Rejection:** 400 INVALID_COMPANY_NUMBER

Business Goal: Display the official UK company registration number on legal documents.

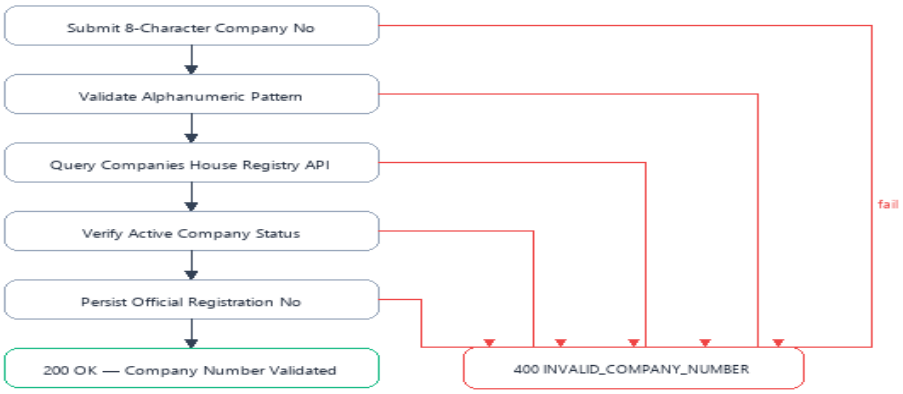


Figure 014 — UK Companies House Registration Number Validation Flow Architecture

How the System Works: 1. User enters their 8-character Companies House number (e.g., '12345678' or 'SC123456'). ➔ 2. The system validates the alphanumeric format against official Companies House standards. ➔ 3. The validated number is permanently stored in the company's legal profile. ➔ 4. The number is automatically included in the footer of all generated PDF invoices and statements. ➔ 5. This fulfills the legal disclosure requirements of the UK Companies Act 2006.
Outcome: All customer invoices and credit notes legally comply with UK company disclosure laws. | **Guardrail:** Entering invalid characters shows: 'Please enter a valid 8-character UK company registration number.'

UC-015: Base Trading Currency Lock (GBP)

Module: Accounting Rules | **Actor:** Certified Accountant | **Rejection:** 422 BASE_CURRENCY_LOCKED

Business Goal: Lock the company's primary ledger currency to prevent financial corruption.

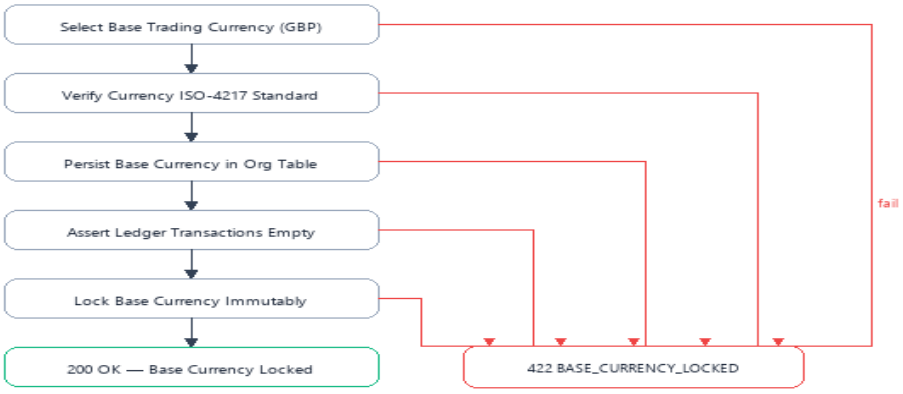


Figure 015 — Base Trading Currency Lock (GBP) Flow Architecture

How the System Works: 1. During setup, the company confirms British Pounds (GBP - £) as its base trading currency. ➔ 2. Blynt sets this currency across all General Ledger asset, liability, and equity control accounts. ➔ 3. Once the first financial transaction is posted to the ledger, Blynt permanently locks the base currency. ➔ 4. Any future attempt to change the base currency in settings is blocked. ➔ 5. Multi-currency transactions (USD/EUR) are fully supported, but always revalued into GBP.
Outcome: Financial records remain stable and compliant with UK GAAP without exchange rate distortions. | **Guardrail:** Attempting to alter the base currency mid-year displays: 'Base currency cannot be changed after trading commences.'

UC-016: Financial Year Calendar Setup & Partitioning

Module: Accounting Foundations | **Actor:** Accountant / Business Owner | **Rejection:** 400 INVALID_DATE_RANGE

Business Goal: Establish the 12-month fiscal calendar for tax reporting.

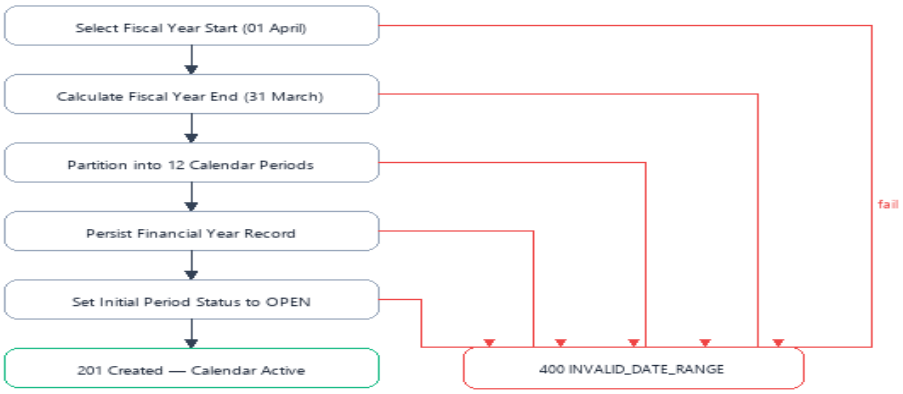


Figure 016 — Financial Year Calendar Setup & Partitioning Flow Architecture

How the System Works: 1. User selects their annual accounting year start date (typically 01 April for UK SME tax year). ➔ 2. Blynt calculates the corresponding year end date (31 March of the following year). ➔ 3. The system automatically partitions the year into 12 chronological monthly accounting periods. ➔ 4. The current period is set to 'OPEN', allowing live bookkeeping entries. ➔ 5. The fiscal calendar is linked to the financial reporting engine for accurate profit-and-loss statements.
Outcome: Accounting periods are cleanly structured for monthly management accounts and year-end tax returns. | **Guardrail:** Selecting overlapping dates displays: 'Accounting periods must be contiguous and non-overlapping.'

UC-017: Company Branding & Invoice Logo Upload

Module: Branding & Customization | **Actor:** Business Owner / Office Staff | **Rejection:** 400 FILE_TOO_LARGE_OR_INVALID

Business Goal: Add professional branding to invoices and customer receipts.



Figure 017 — Company Branding & Invoice Logo Upload Flow Architecture

How the System Works: 1. User uploads their company logo in Settings -> Branding. ➔ 2. The browser inspects the file type, confirming it is an approved PNG, JPEG, or SVG image. ➔ 3. The system verifies that the image size does not exceed 2 Megabytes. ➔ 4. The image is uploaded securely to Blynt's cloud documents storage. ➔ 5. Blynt generates an optimized thumbnail for on-screen display and a high-resolution version for PDF printing.
Outcome: All invoices, credit notes, and customer emails display the company's logo with crisp resolution. | **Guardrail:** Uploading a file larger than 2MB or uploading an unsupported document shows: 'Please upload an image under 2MB.'

UC-018: Organization Soft Deactivation & Archive

Module: Tenancy Governance | **Actor:** Company Owner Only | **Rejection:** 403 OWNER_ROLE_REQUIRED

Business Goal: Safely close a business workspace while keeping historical records for tax audits.

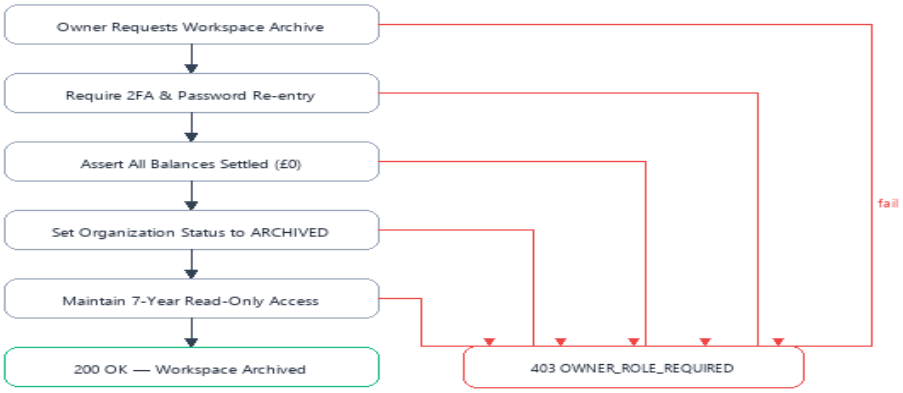


Figure 018 — Organization Soft Deactivation & Archive Flow Architecture

How the System Works: 1. The owner requests to archive an organization that has ceased trading. ➔ 2. The system requires password confirmation and 2FA authentication to prevent accidental closure. ➔ 3. Blynt checks that all invoices and bills are settled and cash balances reconciled. ➔ 4. The workspace status transitions to 'ARCHIVED'—active billing ceases immediately. ➔ 5. All historical ledgers, invoices, and audit logs remain accessible in read-only mode for 7 years (HMRC rule).
Outcome: The business is closed cleanly, billing stops, and tax audit compliance is fully preserved. | **Guardrail:** Non-owner staff attempting to archive receive an immediate 'Only the organization owner can archive a workspace.'

UC-019: Standard Nominal Code Creation (1000-9999)

Module: General Ledger | **Actor:** Accountant / Bookkeeper | **Rejection:** 400 INVALID_CODE_FORMAT

Business Goal: Add a custom account code for specific business expense tracking.

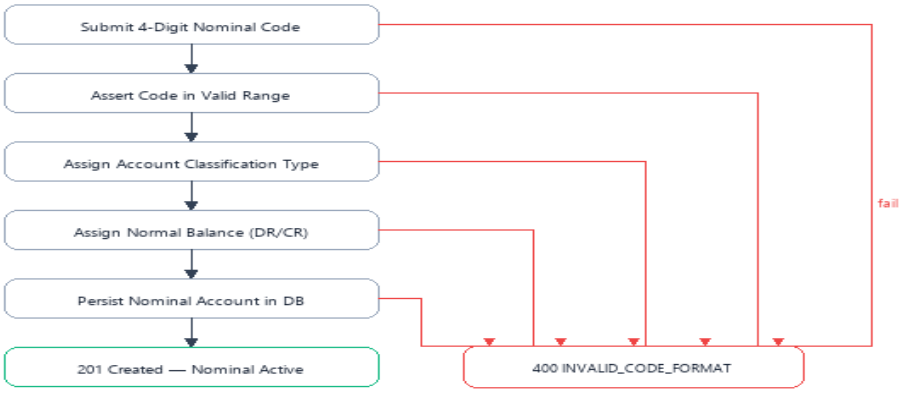


Figure 019 — Standard Nominal Code Creation (1000-9999) Flow Architecture

How the System Works: 1. User goes to Accounting -> Chart of Accounts and clicks '+ New Account'. ➔ 2. Enters a 4-digit code (e.g., '7550'), name ('Software Subscriptions'), and type ('Expense'). ➔ 3. Blynt validates that the code falls within the standard UK GAAP expense range (7000–8999). ➔ 4. The system automatically assigns the normal balance as 'DEBIT'. ➔ 5. The new nominal account appears immediately in dropdowns across invoices and bills.
Outcome: Bookkeepers can categorize business spending into tailored categories matching their operational needs. | **Guardrail:** Typing a code outside allowed ranges displays: 'Account code must be a 4-digit number between 1000 and 9999.'

UC-020: Duplicate Nominal Code Constraint Check

Module: General Ledger | **Actor:** Accountant | **Rejection:** 409 DUPLICATE_ACCOUNT_CODE

Business Goal: Prevent confusion caused by multiple accounts sharing the same number.

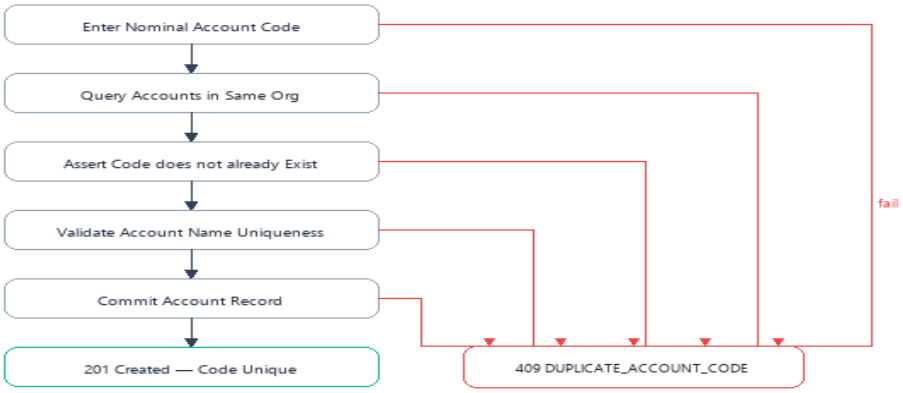


Figure 020 — Duplicate Nominal Code Constraint Check Flow Architecture

How the System Works: 1. User attempts to create an account using code '1200'. ➔ 2. Blynt queries the organization's existing chart of accounts. ➔ 3. The system detects that code '1200' is already assigned to 'Bank Current Account'. ➔ 4. The save operation is rejected before touching the database. ➔ 5. A clear warning highlights the code field, prompting the user to pick an unused number.
Outcome: The General Ledger maintains a strictly unique, clean nominal numbering structure. | **Guardrail:** The screen shows: 'Account code 1200 is already in use by Bank Current Account. Please choose another code.'

UC-021: Accounts Receivable Control Account Protection

Module: Financial Safety | **Actor:** Any Staff Member | **Rejection:** 422 AR_CONTROL_PROTECTED

Business Goal: Prevent manual journal entries from throwing customer balances out of sync.

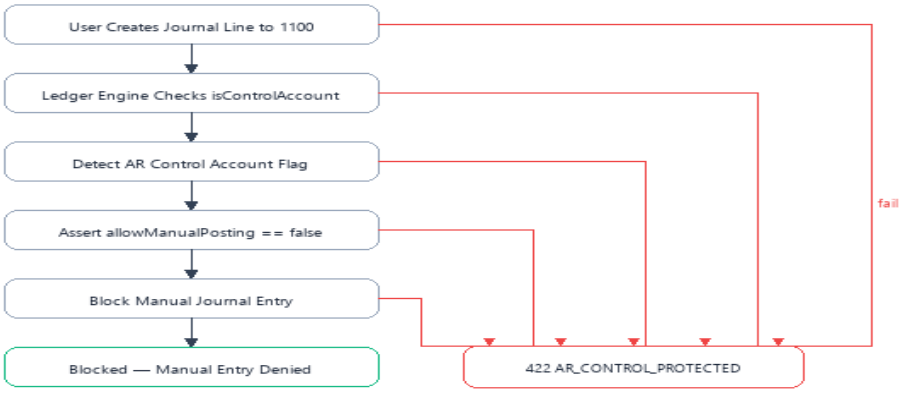


Figure 021 — Accounts Receivable Control Account Protection Flow Architecture

How the System Works: 1. A user creates a manual journal entry and attempts to select Code 1100 (Accounts Receivable). ➔ 2. Blynt's journal engine checks the 'isControlAccount' flag on Code 1100. ➔ 3. The system identifies that 1100 is reserved exclusively for the Sales Invoicing engine. ➔ 4. Manual postings are blocked; the account appears disabled in the manual journal dropdown. ➔ 5. The user is guided to issue a customer credit note or adjustment invoice instead.
Outcome: Customer statements and the general ledger remain 100% synchronized with zero manual discrepancies. | **Guardrail:** Attempting to force a manual entry displays: 'Accounts Receivable is a protected control account. Please use Invoicing.'

UC-022: Accounts Payable Control Account Protection

Module: Financial Safety | **Actor:** Any Staff Member | **Rejection:** 422 AP_CONTROL_PROTECTED

Business Goal: Prevent manual journal entries from corrupting supplier balances.

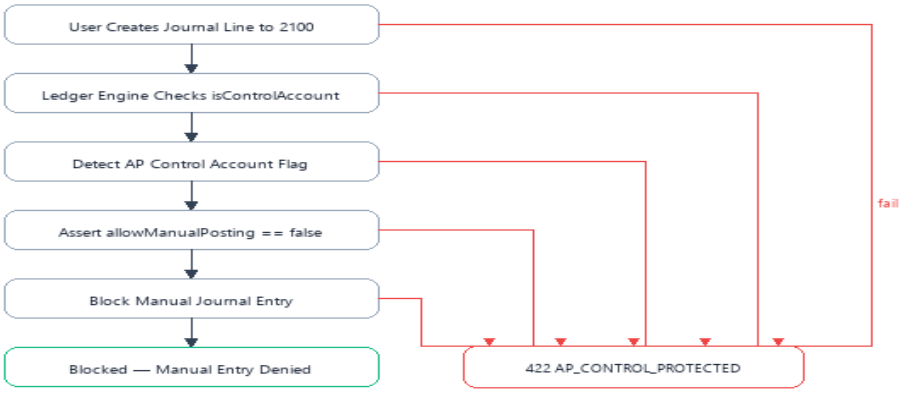


Figure 022 — Accounts Payable Control Account Protection Flow Architecture

How the System Works: 1. A user creates a manual journal entry and attempts to select Code 2100 (Accounts Payable). ➔ 2. Blynt's journal engine checks the 'isControlAccount' flag on Code 2100. ➔ 3. The system identifies that 2100 is reserved exclusively for the Supplier Bills engine. ➔ 4. Manual journal posting is refused. ➔ 5. The user is guided to record a supplier credit note instead.
Outcome: Supplier balances in Accounts Payable always perfectly mirror actual supplier bills and payments. | **Guardrail:** The screen displays: 'Accounts Payable is a protected control account. Please record a supplier bill or credit note.'

UC-023: VAT Control Account Automated Posting Rule

Module: Tax Compliance | **Actor:** Any Staff Member | **Rejection:** 422 VAT_CONTROL_PROTECTED

Business Goal: Ensure VAT liability is only created by verified sales and purchases.

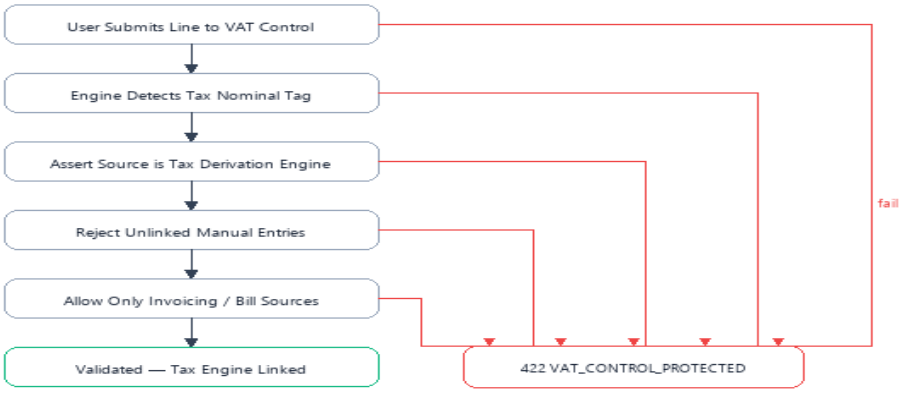


Figure 023 — VAT Control Account Automated Posting Rule Flow Architecture

How the System Works: 1. A user attempts to post a manual journal directly to Code 2200 (VAT Output Tax). ➔ 2. The ledger engine verifies whether the transaction originates from the statutory tax calculation engine. ➔ 3. Direct unlinked manual adjustments to tax liability are blocked. ➔ 4. This prevents accidental distortion of figures intended for quarterly HMRC VAT returns. ➔ 5. Adjustments must be made through approved VAT scheme adjustment workflows.
Outcome: The VAT Return boxes 1 to 9 always reconcile directly to verifiable invoice lines. | **Guardrail:** The screen displays: 'VAT control accounts can only be updated through verified invoicing or tax workflows.'

UC-024: Retained Earnings System Account Lock

Module: Financial Integrity | **Actor:** Any Staff Member | **Rejection:** 422 RETAINED_EARNINGS_LOCKED

Business Goal: Prevent users from tampering with historical accumulated profits.

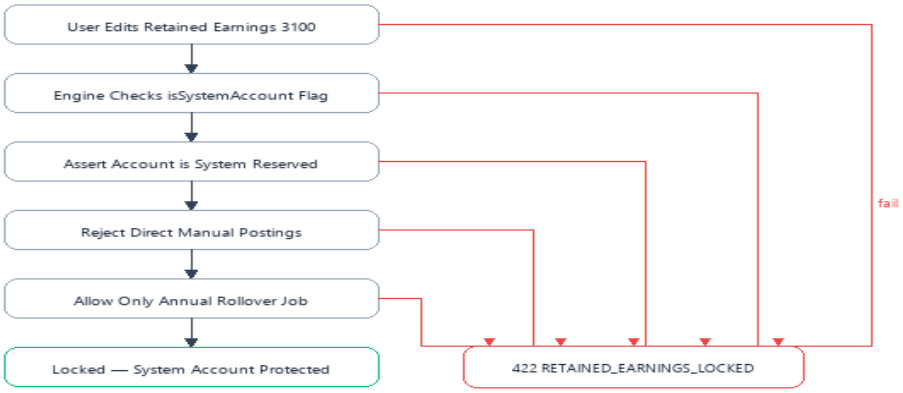


Figure 024 — Retained Earnings System Account Lock Flow Architecture

How the System Works: 1. A user navigates to the Chart of Accounts and attempts to edit Code 3100 (Retained Earnings). ➔ 2. The system checks the 'isSystemAccount' flag. ➔ 3. The account is locked against direct edits, renaming, or manual debit/credit journal entries. ➔ 4. Code 3100 is reserved exclusively for the automated Year-End profit/loss rollover calculation. ➔ 5. All balance changes must be generated organically by closing financial years.
Outcome: Historical business profits are protected from accidental tampering or erasure. | **Guardrail:** The screen displays: 'Retained Earnings is a system-reserved account and cannot be manually modified.'

UC-025: Account Normal Balance Derivation Rule

Module: Accounting Fundamentals | **Actor:** Certified Accountant | **Rejection:** 400 INVALID_NORMAL_BALANCE

Business Goal: Enforce standard double-entry bookkeeping rules for account classification.

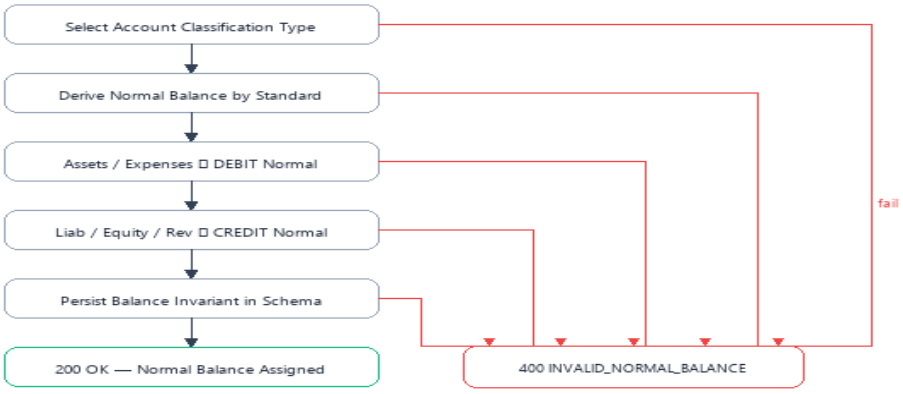


Figure 025 — Account Normal Balance Derivation Rule Flow Architecture

How the System Works: 1. User creates a new nominal account and selects its classification (e.g., 'Current Asset'). ➔ 2. The system applies fundamental accounting logic: Assets & Expenses default to a NORMAL DEBIT balance. ➔ 3. Liabilities, Equity, and Revenue default to a NORMAL CREDIT balance. ➔ 4. Financial reports use this rule to display positive numbers cleanly rather than confusing minus signs. ➔ 5. The classification invariant is permanently stored with the account.
Outcome: Financial reports read intuitively for business owners while maintaining strict accounting rigor. | **Guardrail:** Selecting an invalid classification combination displays a helpful correction tip.

UC-026: Parent-Child Nominal Account Hierarchy

Module: Chart of Accounts | **Actor:** Accountant | **Rejection:** 404 PARENT_NOT_FOUND

Business Goal: Group detailed sub-accounts under main summary headings on reports.

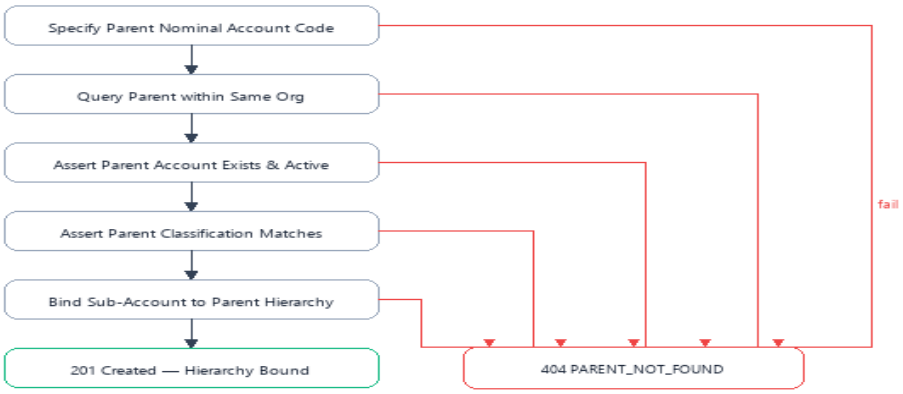


Figure 026 — Parent-Child Nominal Account Hierarchy Flow Architecture

How the System Works: 1. User creates a detailed sub-account (e.g., '7110 - Staff Travel - Rail'). ➔ 2. User selects the parent account ('7100 - Travel & Subsistence'). ➔ 3. Blynt verifies that the parent account exists and shares the exact same classification ('Expense'). ➔ 4. The sub-account is nested under the parent in the Chart of Accounts view. ➔ 5. Financial reports allow expanding or collapsing the parent account for clean summary presentations.
Outcome: Management reports can be viewed at high-level summary or drill-down detail. | **Guardrail:** Attempting to link an Expense account under an Asset parent displays: 'Sub-account classification must match parent.'

UC-027: Account Deactivation with Zero Balance Check

Module: Ledger Cleanliness | **Actor:** Accountant | **Rejection:** 422 ACCOUNT_BALANCE_NON_ZERO

Business Goal: Retire unused nominal accounts without hiding active company money.

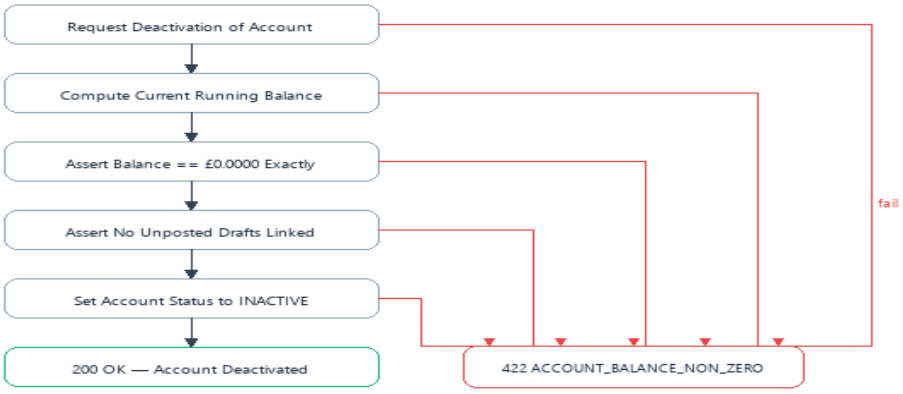


Figure 027 — Account Deactivation with Zero Balance Check Flow Architecture

How the System Works: 1. User selects an old nominal account and clicks 'Deactivate Account'. ➔ 2. Blynt calculates the current running balance of the account across all past journal lines. ➔ 3. If the balance is exactly £0.00, the account is marked 'INACTIVE'. ➔ 4. Inactive accounts disappear from daily invoice and bill dropdowns to keep menus clean. ➔ 5. Historical reports still show the account's past figures for tax audits.
Outcome: The daily bookkeeping interface remains clean and clutter-free. | **Guardrail:** If the account has an active balance of £150.00, deactivation is blocked: 'Cannot deactivate account with a non-zero balance.'

UC-028: Account Foreign Currency Assignment Check

Module: Multi-Currency | **Actor:** Accountant | **Rejection:** 400 UNSUPPORTED_CURRENCY

Business Goal: Track foreign currency bank accounts or supplier balances accurately.

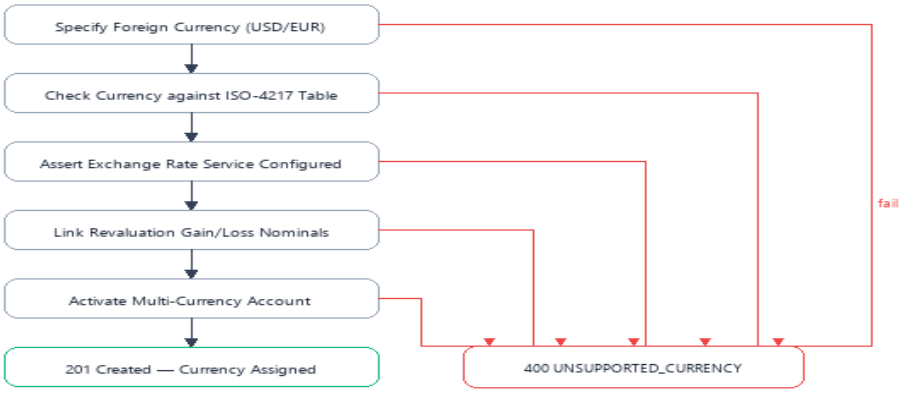


Figure 028 — Account Foreign Currency Assignment Check Flow Architecture

How the System Works: 1. User creates a new bank account designated in US Dollars (USD). ➔ 2. Blynt validates 'USD' against official ISO-4217 international currency standards. ➔ 3. The system links the account to the daily exchange rate service. ➔ 4. Nominals for Realised and Unrealised FX Gain/Loss are automatically linked for revaluations. ➔ 5. The account is ready to receive foreign currency wires and track revaluations back to GBP.
Outcome: Businesses can trade internationally with automatic exchange rate tracking. | **Guardrail:** Entering an unsupported currency code shows: 'Please select a valid international currency code.'

UC-029: Draft Journal Entry Creation & Line Staging

Module: Journal Lifecycle | **Actor:** Accountant | **Rejection:** 400 EMPTY_JOURNAL_PAYLOAD

Business Goal: Prepare complex adjusting entries and review before committing to the books.

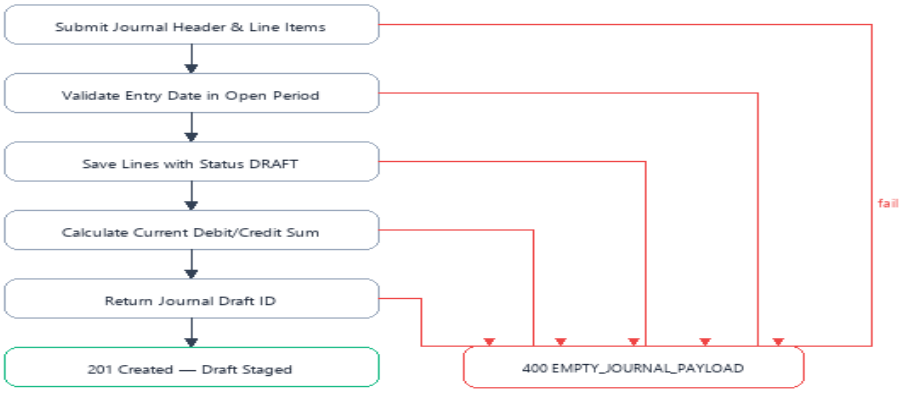


Figure 029 — Draft Journal Entry Creation & Line Staging Flow Architecture

How the System Works: 1. Accountant opens Accounting -> Manual Journals and clicks '+ New Journal'. ➔ 2. Enters a reference note (e.g., 'Month-end depreciation adjustment') and sets the date. ➔ 3. Adds debit and credit lines across multiple asset and expense nominal codes. ➔ 4. Clicks 'Save as Draft'. ➔ 5. The journal is saved in staging mode with status 'DRAFT'—it does NOT alter general ledger balances yet.
Outcome: Accountants can prepare complex year-end entries in advance and review them before posting. | **Guardrail:** Submitting an empty journal displays: 'Please add at least two line items before saving.'

UC-030: Debits Equal Credits Equilibrium Invariant

Module: Accounting Invariants | **Actor:** System Posting Engine | **Rejection:** 422 JOURNAL_UNBALANCED

Business Goal: The fundamental rule of accounting: books can never be out of balance.

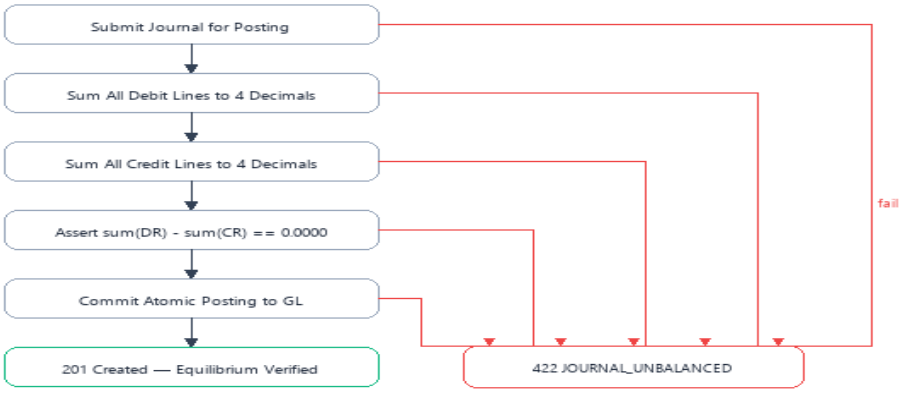


Figure 030 — Debits Equal Credits Equilibrium Invariant Flow Architecture

How the System Works: 1. Accountant reviews a journal entry and clicks 'Post to General Ledger'. ➔ 2. The posting engine sums all Debit lines to 4 decimal places. ➔ 3. The posting engine sums all Credit lines to 4 decimal places. ➔ 4. Blynt asserts that total Debits minus total Credits equals exactly 0.0000. ➔ 5. If balanced, the transaction is committed atomically to the permanent ledger.
Outcome: The General Ledger remains 100% in balance; corrupt or one-sided accounting entries are physically impossible. | **Guardrail:** If debits are £500 and credits are £400, posting is refused: 'Total debits (£500) must equal total credits (£400).'

UC-031: Minimum Two Line Items Enforcement

Module: Accounting Invariants | **Actor:** System Posting Engine | **Rejection:** 400 MINIMUM_LINES_REQUIRED

Business Goal: Ensure every financial entry has both a source and a destination.

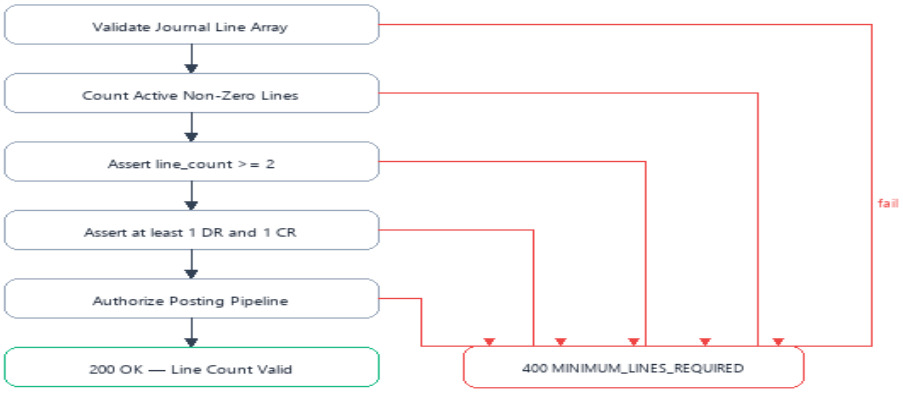


Figure 031 — Minimum Two Line Items Enforcement Flow Architecture

How the System Works: 1. A user attempts to save a journal entry containing only one line (£200 to Bank). ➔ 2. The validation engine counts the active line items. ➔ 3. The system detects that line count is less than 2. ➔ 4. Posting is blocked immediately. ➔ 5. A helper prompt appears: 'A double-entry transaction requires at least one Debit and one Credit line.'
Outcome: Every financial transaction records both where money came from and where it went. | **Guardrail:** The screen highlights: 'Journal entry must contain at least two line items.'

UC-032: Zero Amount Line Item Rejection

Module: Journal Validation | **Actor:** System Posting Engine | **Rejection:** 400 ZERO_AMOUNT_REJECTED

Business Goal: Prevent cluttering the database with empty £0.00 lines.

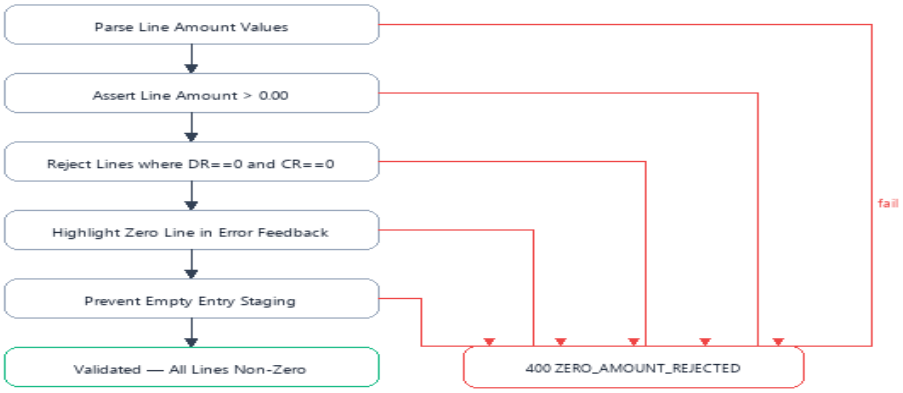


Figure 032 — Zero Amount Line Item Rejection Flow Architecture

How the System Works: 1. A user adds a journal line but leaves the debit and credit amount fields as £0.00. ➔ 2. Upon clicking Save or Post, the validation engine checks each line's monetary value. ➔ 3. The system detects the line has zero monetary value. ➔ 4. The empty line is highlighted in red, or automatically pruned from the payload. ➔ 5. Only lines with real financial value are sent to the database.
Outcome: The General Ledger database remains clean and free of junk entries. | **Guardrail:** The screen shows: 'Line item amounts must be greater than zero.'

UC-033: Mutually Exclusive Line Amounts (DR vs CR)

Module: Journal Validation | **Actor:** System Posting Engine | **Rejection:** 400 MUTUALLY_EXCLUSIVE_LINE

Business Goal: Prevent confusing lines that try to be both money in and money out.

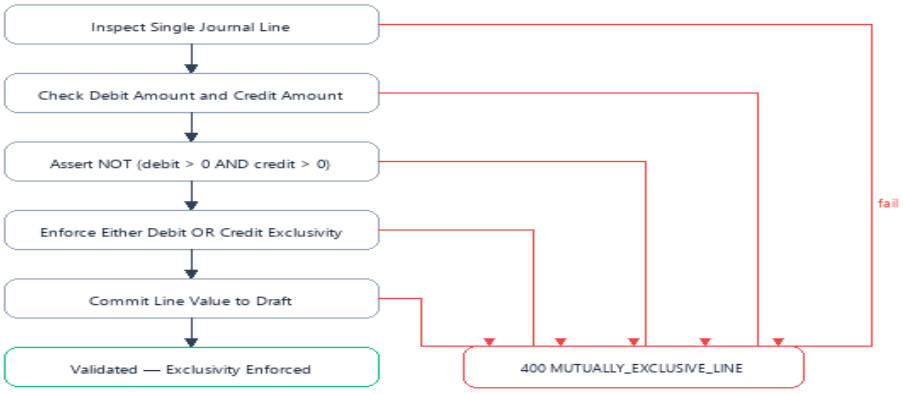


Figure 033 — Mutually Exclusive Line Amounts (DR vs CR) Flow Architecture

How the System Works: 1. A user types £100 in the Debit column and £50 in the Credit column on the SAME line. ➔ 2. The validation engine inspects each line independently. ➔ 3. The system asserts that a line cannot have values in both Debit and Credit columns simultaneously. ➔ 4. The conflicting field is cleared, and an inline warning appears. ➔ 5. The user must place opposing values on separate lines.
Outcome: Accounting lines remain crystal clear for statutory tax inspection. | **Guardrail:** The screen displays: 'A journal line cannot contain both a debit and credit amount. Please use separate lines.'

UC-034: Arbitrary Decimal Precision Arithmetic Check

Module: Financial Precision | **Actor:** Blynt Math Engine | **Rejection:** 422 PRECISION_ROUNDING_ERR

Business Goal: Eliminate fractional rounding errors that cause pennies to disappear.

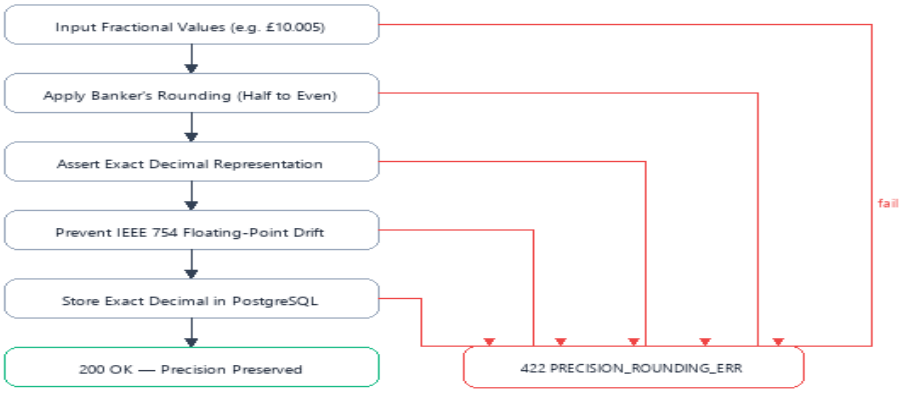


Figure 034 — Arbitrary Decimal Precision Arithmetic Check Flow Architecture

How the System Works: 1. The system computes tax, discounts, or exchange rates that produce fractional pence (e.g., £12.455). ➔ 2. Instead of standard JavaScript floating-point math (which causes rounding bugs), Blynt uses arbitrary-precision decimals. ➔ 3. The engine applies standard UK Banker's Rounding (Round Half to Even). ➔ 4. Numbers are stored in PostgreSQL using the exact NUMERIC(15,4) data type. ➔ 5. Every single penny is accounted for across thousands of transactions.
Outcome: Ledgers balance to the exact penny with zero mathematical drift or rounding leaks. | **Guardrail:** Calculations that fail rounding validation are caught and re-balanced before saving.

UC-035: Atomic Multi-Line Ledger Commit Transaction

Module: Database Integrity | **Actor:** PostgreSQL Engine | **Rejection:** 500 ATOMIC_POST_FAILED

Business Goal: Ensure multi-line transactions succeed together or fail together completely.

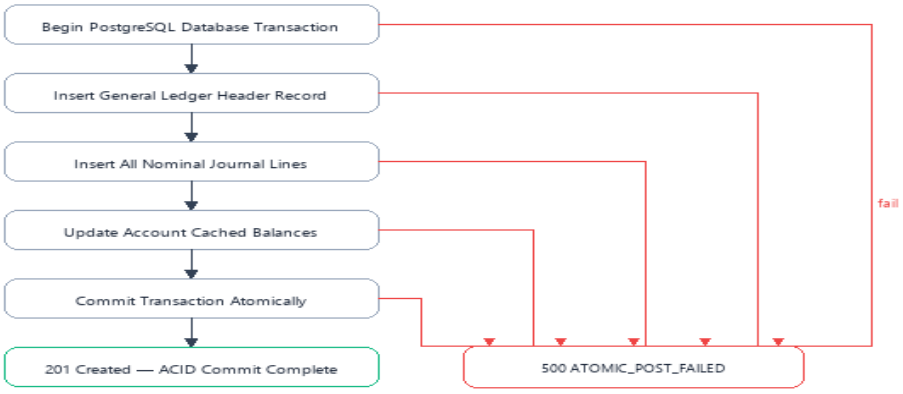


Figure 035 — Atomic Multi-Line Ledger Commit Transaction Flow Architecture

How the System Works: 1. A balanced 10-line journal entry is submitted for posting. ➔ 2. Blynt opens a single atomic PostgreSQL database transaction (BEGIN TRANSACTION). ➔ 3. Lines 1 through 10 are written to the General Ledger table in sequence. ➔ 4. If a database error occurs on line 9 (e.g., network timeout), the transaction triggers an instant ROLLBACK. ➔ 5. Lines 1 through 8 are completely erased from the database as if they never happened.
Outcome: The database never contains half-saved, broken, or corrupt accounting entries. | **Guardrail:** The user sees a clean error notification, and their draft remains intact for resubmission.

UC-036: Posted Journal Historical Immutability

Module: Statutory Compliance | **Actor:** Any User | **Rejection:** 403 POSTED_JOURNAL_IMMUTABLE

Business Goal: Prevent illegal back-room alteration of posted company accounts.

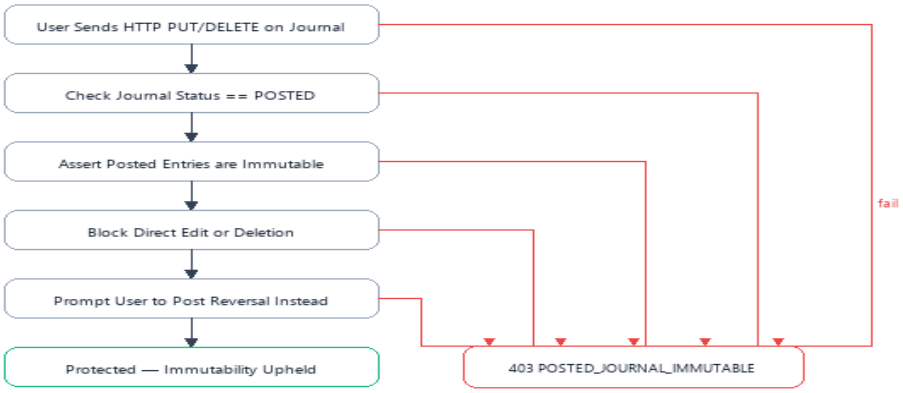


Figure 036 — Posted Journal Historical Immutability Flow Architecture

How the System Works: 1. A user opens a journal entry that was posted last month and looks for an 'Edit' or 'Delete' button. ➔ 2. The system detects that the journal status is 'POSTED'. ➔ 3. All input fields are rendered read-only, and Edit/Delete buttons are completely absent. ➔ 4. Any direct API request to update the row is blocked by database constraints. ➔ 5. The user is guided to click 'Reverse Journal' to post an official correcting entry.
Outcome: Blynt complies 100% with UK HMRC and Companies Act rules against rewriting accounting history. | **Guardrail:** Attempting to force an edit displays: 'Posted accounting transactions are immutable and cannot be edited.'

UC-037: Symmetrical GAAP Journal Reversal Linking

Module: Error Correction | **Actor:** Certified Accountant | **Rejection:** 409 ALREADY_REVERSED

Business Goal: Correct a posted mistake with a clean, linked paper trail.

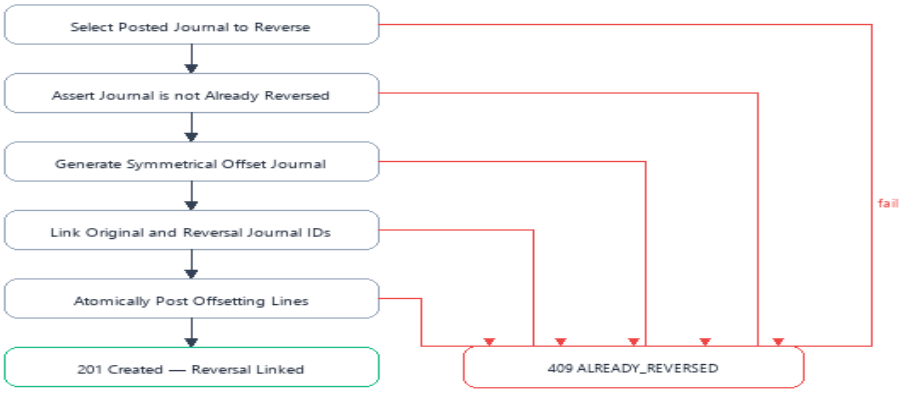


Figure 037 — Symmetrical GAAP Journal Reversal Linking Flow Architecture

How the System Works: 1. Accountant opens a mistaken posted journal and clicks 'Reverse Journal'. ➔ 2. The system asks for an audit reason (e.g., 'Accidental duplicate entry'). ➔ 3. Blynt automatically generates an exact symmetrical copy: original Debits become Credits, Credits become Debits. ➔ 4. The new reversal journal is linked directly to the original transaction ID in the database. ➔ 5. Both entries post to the ledger, perfectly canceling out the financial impact to £0.00.
Outcome: Mistakes are corrected cleanly with zero audit risk; tax inspectors can trace the full story. | **Guardrail:** Attempting to reverse an entry that was already reversed displays: 'This transaction has already been reversed.'

UC-038: Draft Journal Clean Discard & Garbage Collection

Module: Journal Lifecycle | **Actor:** Accountant | **Rejection:** 403 CANNOT_DELETE_POSTED

Business Goal: Clean up unfinished drafts without leaving orphan records.

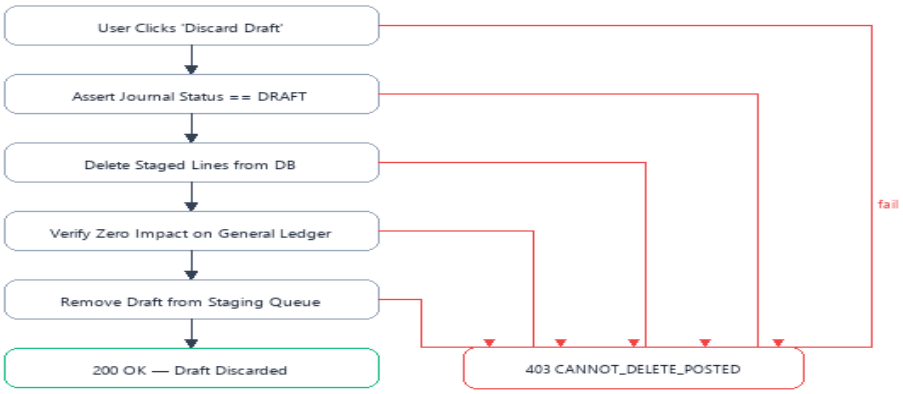


Figure 038 — Draft Journal Clean Discard & Garbage Collection Flow Architecture

How the System Works: 1. Accountant opens an unposted draft journal that is no longer needed. ➔ 2. Clicks 'Discard Draft'. ➔ 3. The system confirms the journal has status 'DRAFT' and was never committed to the General Ledger. ➔ 4. The staged draft lines are removed cleanly from the database. ➔ 5. Ledger balances and account summaries remain completely untouched.
Outcome: Unwanted draft worksheets are cleared without cluttering the accounting workspace. | **Guardrail:** Attempting to delete a posted entry is strictly blocked.

UC-039: Customer Contact Profile Creation & Verification

Module: Commercial Subledgers | **Actor:** Billing Clerk / Sales Team | **Rejection:** 400 INVALID_CONTACT_DATA

Business Goal: Add a client to the directory with complete billing and credit details.

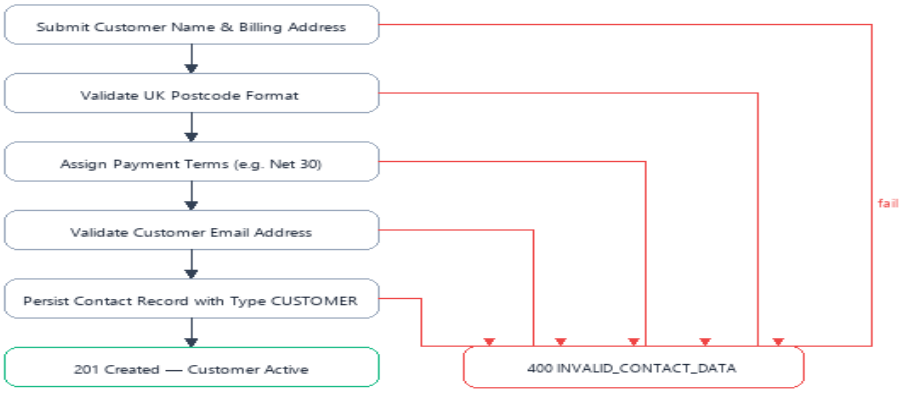


Figure 039 — Customer Contact Profile Creation & Verification Flow Architecture

How the System Works: 1. User opens Contacts -> Customers and clicks '+ New Customer'. ➔ 2. Enters legal business name, contact person, email address, and UK billing address with postcode. ➔ 3. Selects default payment terms (e.g., 'Net 30 Days') and sets an optional credit limit. ➔ 4. The system validates email syntax and ensures the contact is categorized as 'CUSTOMER'. ➔ 5. The customer profile is saved and immediately available for invoicing.
Outcome: Sales invoices can be generated quickly with pre-filled billing terms and addresses. | **Guardrail:** Submitting an invalid email displays: 'Please enter a valid email address.'

UC-040: Sales Invoice Draft Generation

Module: Sales Invoicing | **Actor:** Billing Clerk | **Rejection:** 404 CUSTOMER_NOT_FOUND

Business Goal: Start a new invoice for services delivered.

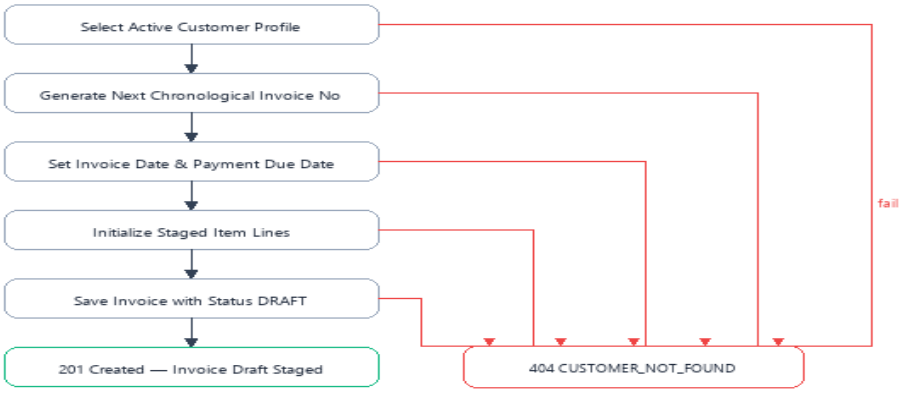


Figure 040 — Sales Invoice Draft Generation Flow Architecture

How the System Works: 1. User clicks Sales -> Invoices -> '+ New Invoice'. ➔ 2. Selects the customer from the searchable dropdown. ➔ 3. Blynt automatically assigns the next sequential invoice number (e.g., 'INV-1042'). ➔ 4. Sets the invoice issue date and auto-calculates the due date based on the customer's payment terms. ➔ 5. The invoice is saved as a 'DRAFT', ready for line items.
Outcome: Invoice numbers remain strictly sequential with zero missing numbers or duplicates. | **Guardrail:** Selecting an archived customer displays: 'Cannot invoice an archived customer. Please reactivate first.'

UC-041: UK Standard 20% VAT Line Derivation

Module: Tax Derivation | **Actor:** Billing Clerk | **Rejection:** 400 INVALID_TAX_CALCULATION

Business Goal: Calculate standard UK VAT on goods and professional services.

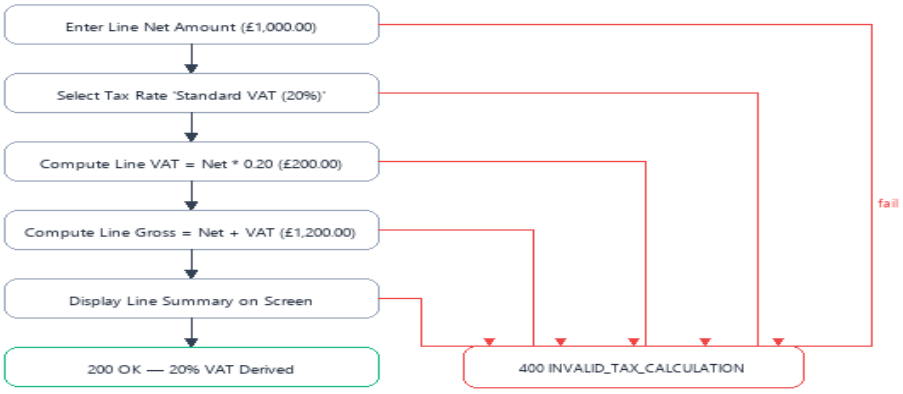


Figure 041 — UK Standard 20% VAT Line Derivation Flow Architecture

How the System Works: 1. User adds an item line: 'Consulting Services', Quantity 1, Unit Price £1,000.00. ➔ 2. Selects tax rate 'Standard VAT (20%)'. ➔ 3. Blynt multiplies £1,000.00 by 0.20 to derive exact VAT of £200.00. ➔ 4. The line total computes as £1,200.00 gross. ➔ 5. The VAT amount is automatically routed to Output Tax nominal Code 2200 for VAT returns.
Outcome: VAT is calculated with mathematical precision, preventing costly HMRC tax return errors. | **Guardrail:** Manually overriding tax with invalid math triggers an instant correction prompt.

UC-042: UK Reduced 5% VAT Line Derivation

Module: Tax Derivation | **Actor:** Billing Clerk | **Rejection:** 400 INVALID_TAX_RATE

Business Goal: Apply qualifying reduced VAT on energy or domestic products.

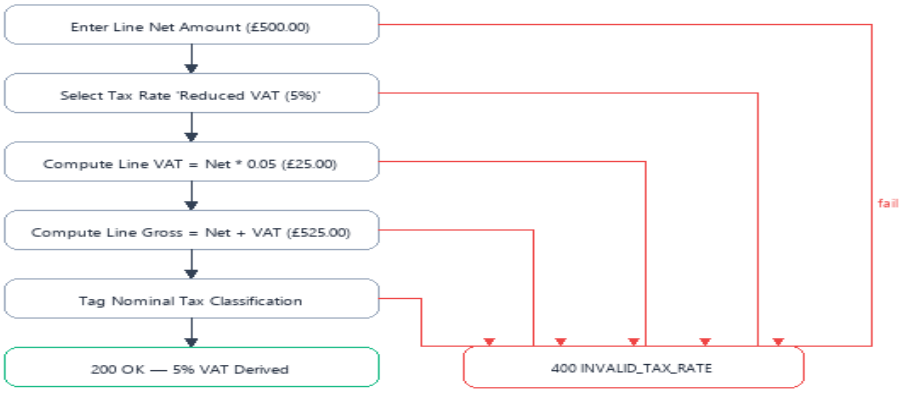


Figure 042 — UK Reduced 5% VAT Line Derivation Flow Architecture

How the System Works: 1. User adds a line for qualifying goods (e.g., energy-saving materials), Net Price £500.00. ➔ 2. Selects tax rate 'Reduced VAT (5%)'. ➔ 3. The system applies 5% math: Net £500.00 + VAT £25.00 = Total £525.00. ➔ 4. The line is tagged with the statutory 5% tax code. ➔ 5. The tax amount is tracked separately for Box 1 output reporting.
Outcome: Specialist VAT rates are applied accurately without manual spreadsheet calculations. | **Guardrail:** Selecting an incompatible tax rate displays a clarification tip.

UC-043: Zero-Rated (0%) VAT Classification

Module: Tax Derivation | **Actor:** Billing Clerk | **Rejection:** 400 ZERO_RATE_MISCLASSIFIED

Business Goal: Invoice items that legally carry zero tax (books, children's clothing, food).

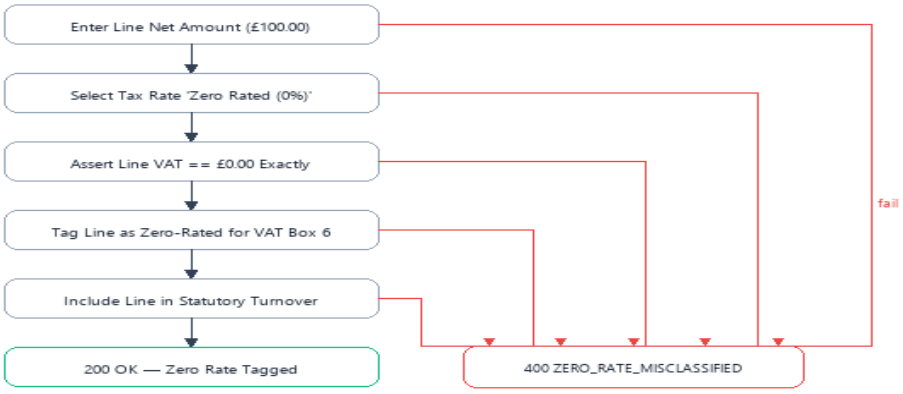


Figure 043 — Zero-Rated (0%) VAT Classification Flow Architecture

How the System Works: 1. User adds an item line for qualifying zero-rated products, Net Price £150.00. ➔ 2. Selects tax rate 'Zero-Rated (0%)'. ➔ 3. The system sets VAT to exactly £0.00, keeping Total at £150.00. ➔ 4. The net amount is tagged to report in VAT Return Box 6 (Total Sales excluding VAT). ➔ 5. No output tax liability is created.
Outcome: Zero-rated sales report properly on statutory returns without inflating tax owed. | **Guardrail:** Misclassifying exempt sales as zero-rated is flagged by the validation engine.

UC-044: VAT Exempt Sales Classification

Module: Tax Derivation | **Actor:** Billing Clerk | **Rejection:** 400 EXEMPT_MISCLASSIFIED

Business Goal: Invoice statutory exempt services (financial services, training).

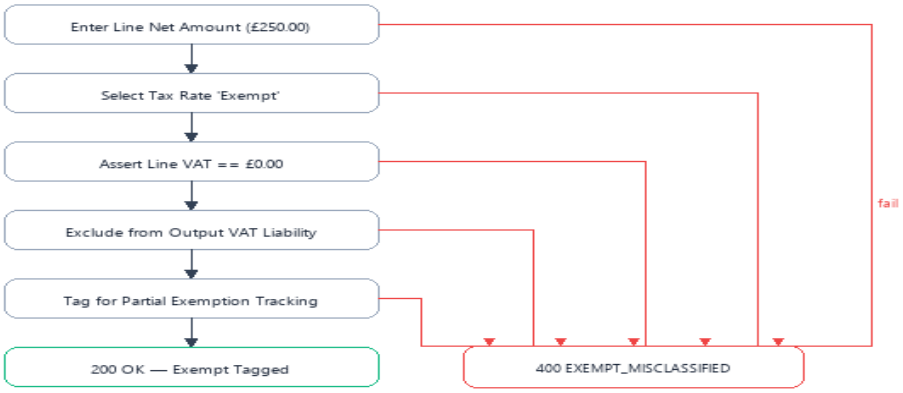


Figure 044 — VAT Exempt Sales Classification Flow Architecture

How the System Works: 1. User adds a line for exempt educational or financial services, Net Price £300.00. ➔ 2. Selects tax rate 'Exempt'. ➔ 3. The system applies £0.00 VAT. ➔ 4. The transaction is tagged as Exempt for partial exemption calculations. ➔ 5. The invoice prints the appropriate legal exemption note.
Outcome: Exempt income is isolated cleanly from taxable turnover for tax accounting. | **Guardrail:** Selecting exempt on standard taxable goods prompts an accountant warning.

UC-045: CIS Domestic Reverse Charge Application

Module: Construction Sector | **Actor:** Construction Contractor | **Rejection:** 400 CIS_REVERSE_CHARGE_ERR

Business Goal: Comply with UK construction industry reverse charge VAT rules.

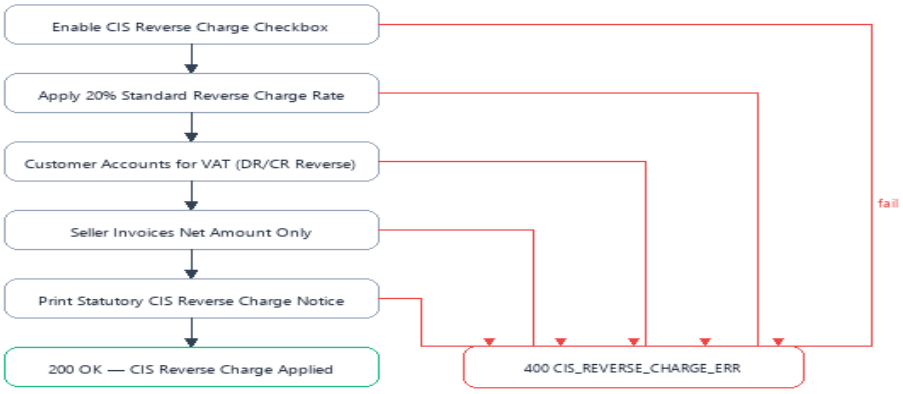


Figure 045 — CIS Domestic Reverse Charge Application Flow Architecture

How the System Works: 1. User checks the 'Apply CIS Domestic Reverse Charge' box on a construction invoice. ➔ 2. The standard 20% VAT rate is selected, but the VAT amount is NOT added to the invoice total payable. ➔ 3. The invoice states: 'Reverse charge applies. Customer must account for the VAT to HMRC.' ➔ 4. The customer pays only the net amount (£1,000.00). ➔ 5. The seller's VAT Return records the sale in Box 6 with zero liability in Box 1.
Outcome: Construction businesses comply automatically with complex HMRC reverse charge laws. | **Guardrail:** Failing to provide the customer's verified VAT number blocks the reverse charge.

UC-046: Invoice Total Arithmetic Validation Gate

Module: Invoice Validation | **Actor:** System Posting Engine | **Rejection:** 422 ARITHMETIC_TOTAL_MISMATCH

Business Goal: Guarantee that invoice totals match line items down to the exact penny.

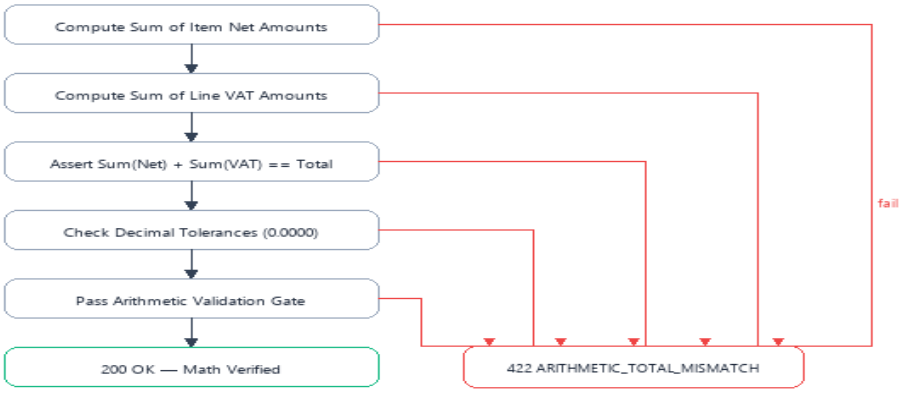


Figure 046 — Invoice Total Arithmetic Validation Gate Flow Architecture

How the System Works: 1. User clicks 'Approve & Issue' on an invoice. ➔ 2. The posting engine sums all line net amounts. ➔ 3. The engine sums all line VAT amounts. ➔ 4. The engine adds the two sums and compares against the invoice totalAmount. ➔ 5. If the values match down to 0.0000 decimal tolerance, the invoice passes the gate.
Outcome: Invoices with arithmetic errors or rounding discrepancies can never be issued to customers. | **Guardrail:** If a manual edit caused a 1p mismatch, the system halts: 'Line totals do not match invoice grand total.'

UC-047: Invoice Approval & General Ledger Posting

Module: Ledger Engine | **Actor:** Billing Clerk / Business Owner | **Rejection:** 400 APPROVAL_VALIDATION_ERR

Business Goal: Lock the invoice and record the revenue in the official books.

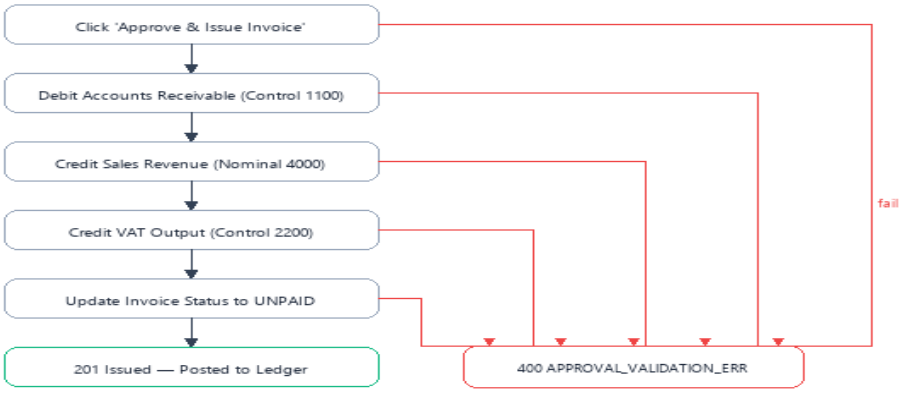


Figure 047 — Invoice Approval & General Ledger Posting Flow Architecture

How the System Works: 1. User approves a validated invoice for £1,200.00 (£1,000 net + £200 VAT). ➔ 2. Blynt atomically generates the balanced General Ledger journal: ➔ 3. - DEBIT Code 1100 (Accounts Receivable) £1,200.00 ➔ 4. - CREDIT Code 4000 (Sales Revenue) £1,000.00 ➔ 5. - CREDIT Code 2200 (VAT Output Tax) £200.00 ➔ 6. The invoice status transitions from 'DRAFT' to 'AWAITING_PAYMENT'.
Outcome: Revenue and tax liability are recorded in real-time, immediately updating the company Profit & Loss. | **Guardrail:** Attempting to approve an invoice with missing customer details displays an inline error.

UC-048: Customer Subledger Balance Synchronization

Module: Commercial Subledgers | **Actor:** Subledger Engine | **Rejection:** 500 SUBLEDGER_SYNC_FAILED

Business Goal: Update what the customer owes the moment an invoice is approved.

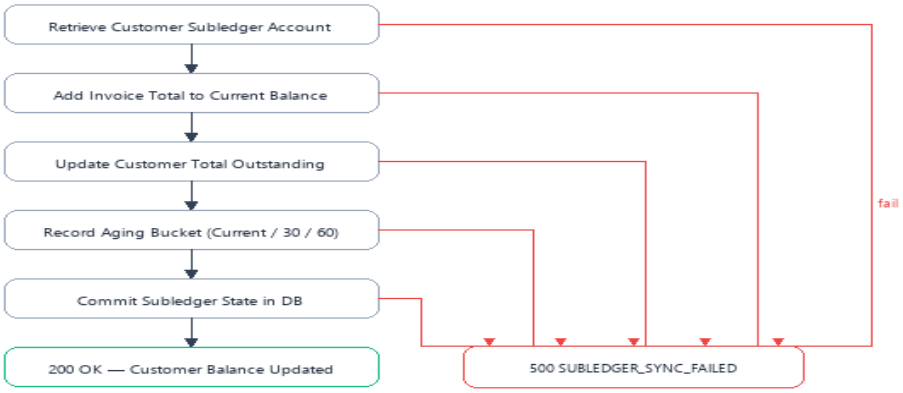


Figure 048 — Customer Subledger Balance Synchronization Flow Architecture

How the System Works: 1. The moment Invoice #1042 for £1,200.00 is approved, the customer subledger engine fires. ➔ 2. Blynt retrieves the customer's account (e.g., 'Acme UK Ltd'). ➔ 3. The customer's current balance increases by exactly £1,200.00. ➔ 4. The invoice is placed in the 'Current (0-30 Days)' aging bucket. ➔ 5. The updated balance is instantly visible on the customer's overview card.
Outcome: Finance teams can see exactly who owes money in real time without running end-of-day batch jobs. | **Guardrail:** If a database lock occurs, the entire transaction rolls back to prevent balance divergence.

UC-049: Customer Statement Chronological Running Balance

Module: Customer Reporting | **Actor:** Credit Controller / Customer | **Rejection:** 200 STATEMENT_GENERATED

Business Goal: Provide clients with a crystal-clear statement showing every bill and payment.

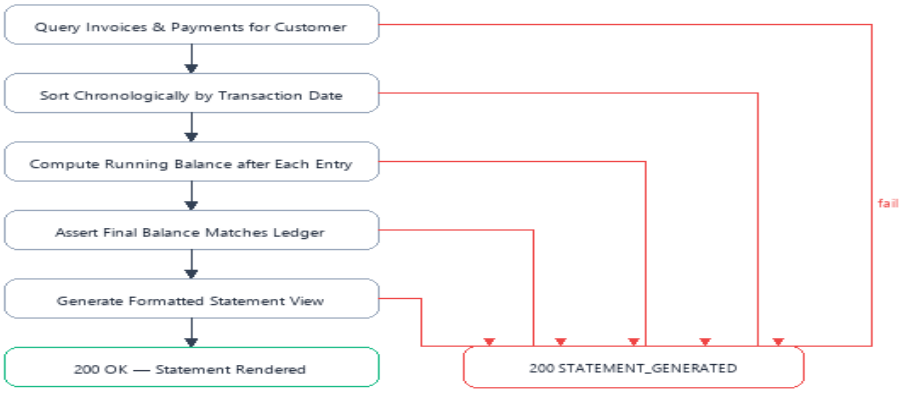


Figure 049 — Customer Statement Chronological Running Balance Flow Architecture

How the System Works: 1. User clicks 'Generate Statement' on a customer profile for a given date range. ➔ 2. Blynt queries all invoices, credit notes, and payments for that customer in date order. ➔ 3. The system computes a running balance after every single transaction line. ➔ 4. The final closing balance matches the customer's current ledger balance to the penny. ➔ 5. The statement can be downloaded as an A4 PDF or sent directly via email.
Outcome: Customers receive a transparent account summary, speeding up payment collection. | **Guardrail:** Statements reflect all cleared receipts and credit notes in exact chronological order.

UC-050: Print-Ready A4 Invoice PDF Compilation

Module: Document Generation | **Actor:** Billing Clerk / Customer | **Rejection:** 500 PDF_RENDER_FAILED

Business Goal: Generate a pixel-perfect, branded PDF ready to email or print.

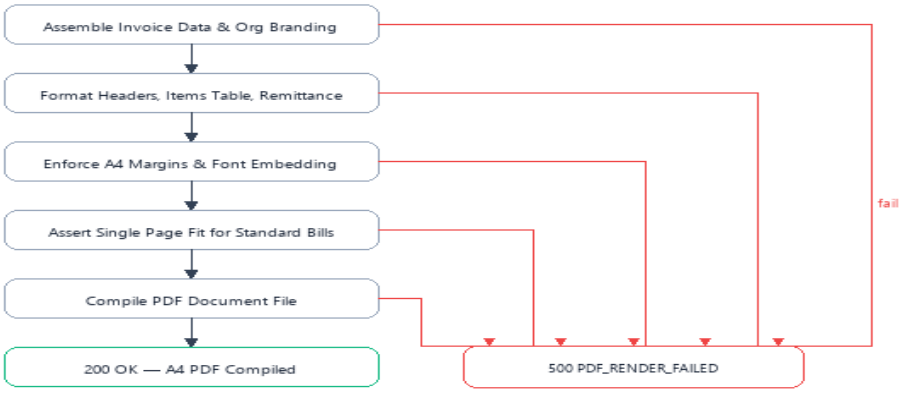


Figure 050 — Print-Ready A4 Invoice PDF Compilation Flow Architecture

How the System Works: 1. User clicks 'Download PDF' on an invoice. ➔ 2. Blynt's PDF engine pulls the company logo, legal address, VAT number, and payment remittance details. ➔ 3. Formats line items into a crisp, readable table with clear tax breakdowns. ➔ 4. Enforces standard A4 dimensions with proper margins so single-page invoices fit on one sheet. ➔ 5. Compiles the PDF and downloads it to the user's computer in under 2 seconds.
Outcome: Clients receive clean, executive-level PDF bills formatted for corporate accounting departments. | **Guardrail:** If an asset is temporarily missing, the engine gracefully falls back to clean text styling.

UC-051: Public Magic Token Invoice View Access

Module: Client Experience | **Actor:** External Customer | **Rejection:** 404 TOKEN_EXPIRED_OR_INVALID

Business Goal: Allow clients to view their bill in a web browser without creating an IT account.

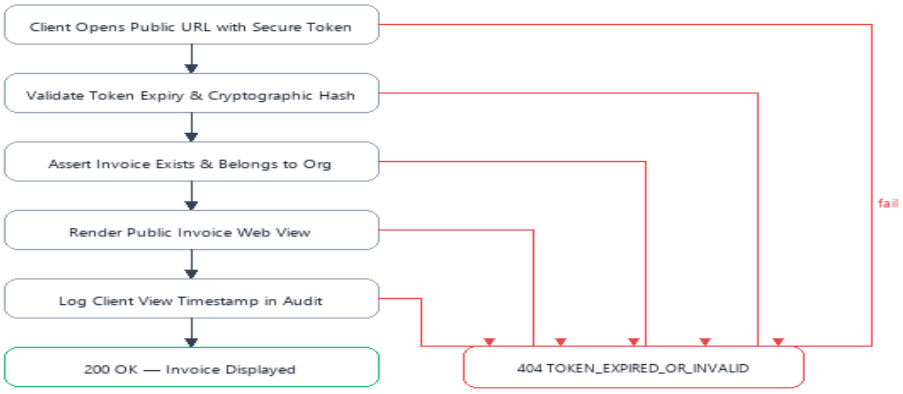


Figure 051 — Public Magic Token Invoice View Access Flow Architecture

How the System Works: 1. When an invoice is issued, Blynt generates an expiring cryptographic access token. ➔ 2. The email to the client includes a link: 'https://app.blynt.co.uk/portal/invoices/{token}'. ➔ 3. The customer clicks the link in their email on their phone or laptop. ➔ 4. Blynt validates the token's signature and confirms it has not expired. ➔ 5. The branded invoice renders cleanly in the client's browser with an active 'Pay Now' button.
Outcome: Clients view their bill with zero friction; no logins, passwords, or IT barriers. | **Guardrail:** Clicking an invalid or expired link shows: 'This invoice link has expired. Please contact the vendor for a fresh link.'

UC-052: Client Portal Brand Consistency Verification

Module: Client Experience | **Actor:** External Customer | **Rejection:** 200 BRANDING_VERIFIED

Business Goal: Give payers confidence they are dealing with the real vendor.

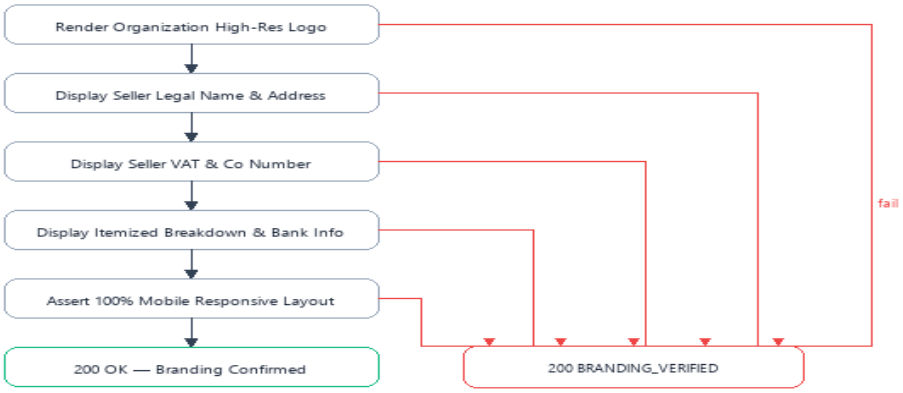


Figure 052 — Client Portal Brand Consistency Verification Flow Architecture

How the System Works: 1. The client opens the online invoice page. ➔ 2. The web page displays the vendor's high-resolution company logo, legal registered address, and phone number. ➔ 3. Company registration number and VAT number are clearly visible at the top. ➔ 4. Bank remittance details (Sort Code, Account Number, Reference) are clearly printed. ➔ 5. The layout adapts automatically to phone, tablet, or desktop screens.
Outcome: Customers trust the payment portal, eliminating phishing concerns and speeding up settlement. | **Guardrail:** Missing branding defaults cleanly to verified company legal text.

UC-053: Online Credit Card Payment Initiation

Module: Payment Gateway | **Actor:** External Customer | **Rejection:** 400 INVALID_PAYMENT_AMOUNT

Business Goal: Enable instant card checkout directly on the invoice screen.

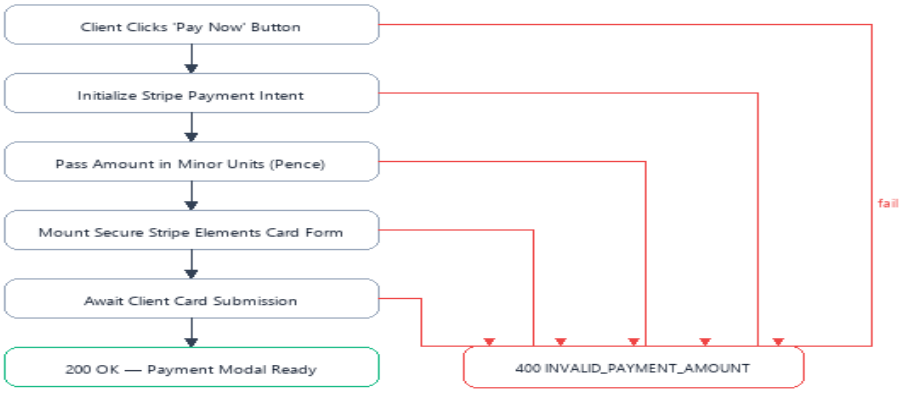


Figure 053 — Online Credit Card Payment Initiation Flow Architecture

How the System Works: 1. Customer clicks the prominent 'Pay £1,200.00' button on their online invoice. ➔ 2. Blynt initializes a secure Stripe Payment Intent for exactly £1,200.00 (120,000 minor units). ➔ 3. A secure Stripe Elements card input modal opens over the invoice. ➔ 4. Customer enters their card number, expiry date, CVC, and billing postcode. ➔ 5. Customer clicks 'Authorize Payment'.
Outcome: Customers can settle invoices in 30 seconds using Visa, Mastercard, or American Express. | **Guardrail:** Entering an invalid card number displays immediate red inline validation from the card network.

UC-054: Payment Idempotency & Double-Click Shield

Module: Client-Side Safety | **Actor:** External Customer with Slow Internet | **Rejection:** 409 DUPLICATE_SUBMISSION_BLOCKED

Business Goal: Guarantee that a customer is never billed twice even if clicking rapidly.

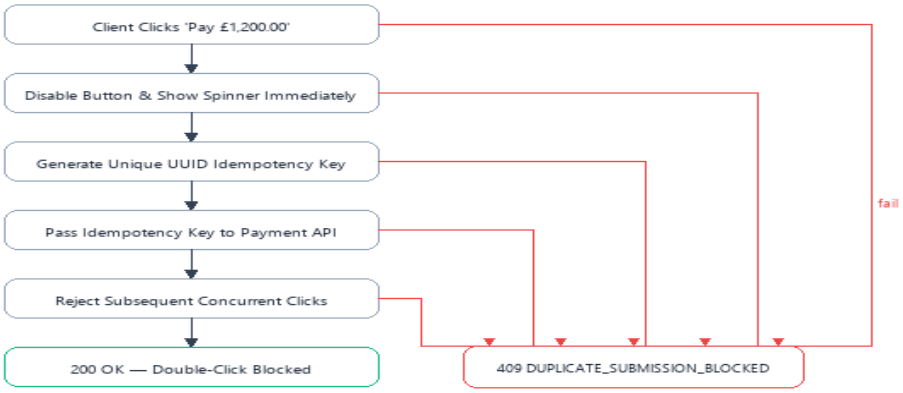


Figure 054 — Payment Idempotency & Double-Click Shield Flow Architecture

How the System Works: 1. A customer with a slow internet connection clicks the 'Pay' button 3 times in quick succession. ➔ 2. On the very first click, Blynt disables the button, turns it grey, and displays a spinner: 'Processing...'. ➔ 3. The browser generates a unique UUID idempotency key attached to the charge request. ➔ 4. The payment gateway recognizes the idempotency key and executes only ONE transaction. ➔ 5. Subsequent rapid clicks are discarded with zero financial impact.
Outcome: The customer is charged exactly once; duplicate billing complaints are completely eliminated. | **Guardrail:** The system safely blocks duplicate submissions with an idempotency lock.

UC-055: Declined Card Friendly Error Handling

Module: Payment Gateway | **Actor:** External Customer | **Rejection:** 402 CARD_DECLINED_BY_BANK

Business Goal: Handle card payment failures gracefully without losing the customer.

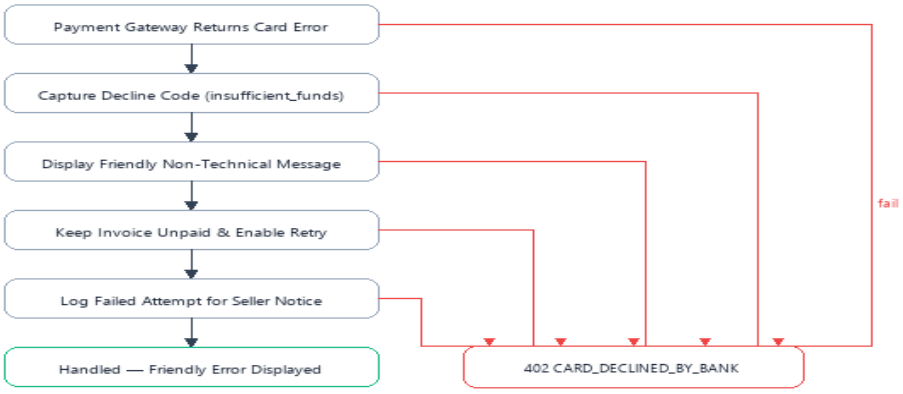


Figure 055 — Declined Card Friendly Error Handling Flow Architecture

How the System Works: 1. Customer submits a card that has expired or has insufficient funds. ➔ 2. The card network returns a decline code (e.g., 'card_declined' or 'insufficient_funds'). ➔ 3. Blynt translates the technical decline into friendly advice: 'Your bank declined this card. Please try another card.' ➔ 4. The modal resets, allowing the customer to type another card number immediately. ➔ 5. The invoice remains open and unpaid; no corrupt records enter the system.
Outcome: Customers understand what went wrong and can resolve it on the spot with another card. | **Guardrail:** The screen shows clear non-technical guidance without exposing internal error codes.

UC-056: Instant Payment Confirmation & Status Flip

Module: Ledger Automation | **Actor:** Automated Webhook Engine | **Rejection:** 500 WEBHOOK_PROCESSING_FAILED

Business Goal: Automatically settle the invoice the second the money is approved.

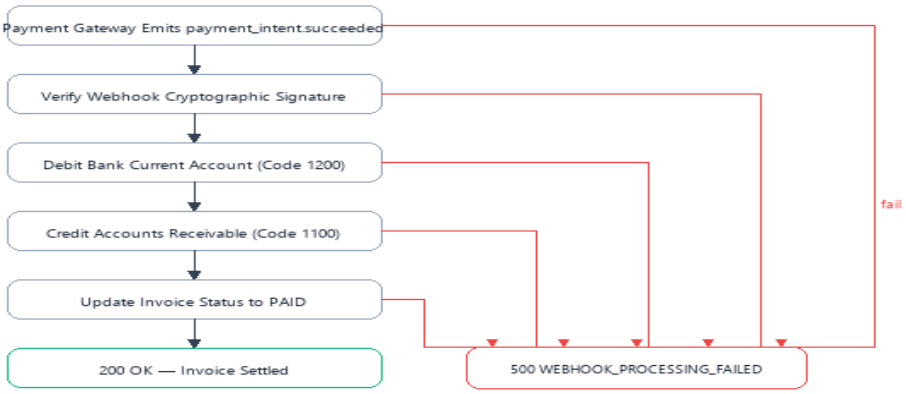


Figure 056 — Instant Payment Confirmation & Status Flip Flow Architecture

How the System Works: 1. The card gateway approves the charge and emits a 'payment_intent.succeeded' webhook. ➔ 2. Blynt validates the cryptographic webhook signature to ensure it came from the real payment provider. ➔ 3. The system automatically posts the cash receipt: DEBIT Bank (1200) £1,200.00, CREDIT AR (1100) £1,200.00. ➔ 4. The invoice status flips from 'AWAITING_PAYMENT' to green 'PAID'. ➔ 5. The customer's screen updates in real-time with a green checkmark: 'Payment Successful!'
Outcome: The invoice is marked paid and cash is recorded with zero manual human bookkeeping required. | **Guardrail:** If the webhook payload is corrupted, it is rejected and queued for automated retry.

UC-057: Automated Client Payment Receipt Delivery

Module: Client Communications | **Actor:** Automated Email Engine | **Rejection:** 200 RECEIPT_DISPATCHED

Business Goal: Provide clients with an instant tax-deductible payment receipt.

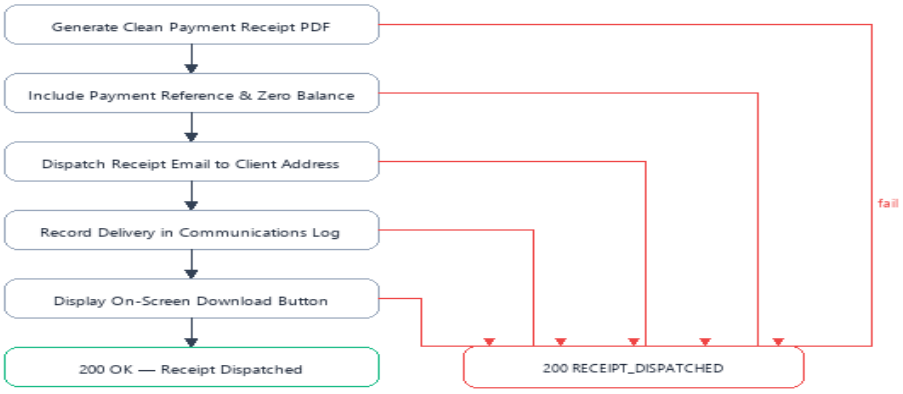


Figure 057 — Automated Client Payment Receipt Delivery Flow Architecture

How the System Works: 1. Immediately following payment confirmation, Blynt generates an official Payment Receipt PDF. ➔ 2. The receipt details the transaction reference, payment method (e.g., Visa ending in 4242), and £0.00 balance. ➔ 3. The system emails the receipt PDF to the customer's billing email address within 10 seconds. ➔ 4. A copy of the receipt is permanently archived in the invoice's activity history. ➔ 5. The customer can also click 'Download Receipt' directly on their browser screen.
Outcome: Clients have immediate proof of payment for their own accounting files. | **Guardrail:** Receipt delivery is verified and logged in the system communication log.

UC-058: Self-Service Client Historical Statements

Module: Customer Self-Service | **Actor:** Corporate Client Finance Team | **Rejection:** 200 CLIENT_PORTAL_SUCCESS

Business Goal: Allow clients to download past invoices without calling customer support.

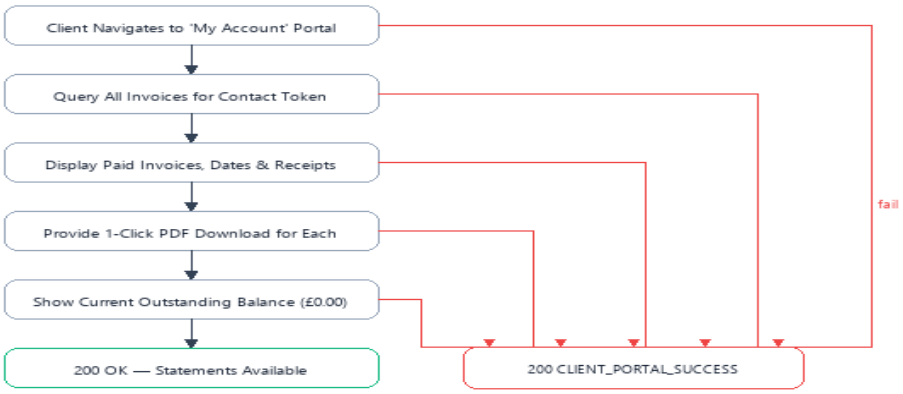


Figure 058 — Self-Service Client Historical Statements Flow Architecture

How the System Works: 1. A corporate client accesses their account history via the link on any previous invoice. ➔ 2. Blynt displays a clean table of all historical invoices issued to that company. ➔ 3. Client can see which invoices are paid, payment dates, and any current outstanding amount. ➔ 4. One-click 'Download PDF' buttons allow downloading any past invoice or receipt. ➔ 5. Client can download a complete annual trading statement for their year-end audit.
Outcome: Reduces customer service billing queries by over 70%. | **Guardrail:** Clients have 24/7 self-service access to their complete financial paperwork.

UC-059: Supplier Contact Profile Setup & Classification

Module: Purchases & AP | **Actor:** Bookkeeper | **Rejection:** 400 INVALID_SUPPLIER_DATA

Business Goal: Set up a new supplier in the vendor registry with banking and payment terms.

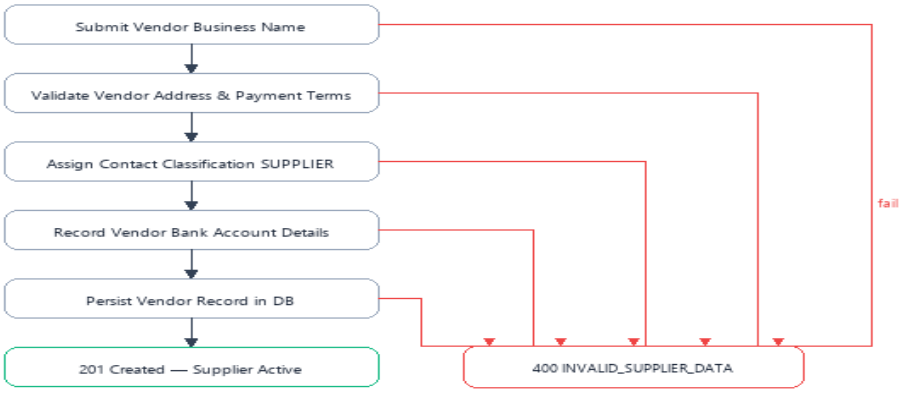


Figure 059 — Supplier Contact Profile Setup & Classification Flow Architecture

How the System Works: 1. Bookkeeper clicks Contacts -> Suppliers -> '+ New Supplier'. ➔ 2. Enters supplier legal business name, address, email, and payment terms (e.g., 'Due in 30 Days'). ➔ 3. Enters the supplier's bank account details (Sort Code and Account Number) for outgoing payments. ➔ 4. The system categorizes the contact as 'SUPPLIER' and validates bank number formats. ➔ 5. The supplier profile is saved and immediately available for bill entry.
Outcome: Supplier payment details are stored safely for accurate electronic bill payments. | **Guardrail:** Submitting missing vendor names shows: 'Please enter the supplier's legal name.'

UC-060: Supplier Bill Entry with Invoice Reference

Module: Purchases & AP | **Actor:** Bookkeeper | **Rejection:** 400 MISSING_SUPPLIER_REF

Business Goal: Record an incoming vendor bill against the correct expense categories.

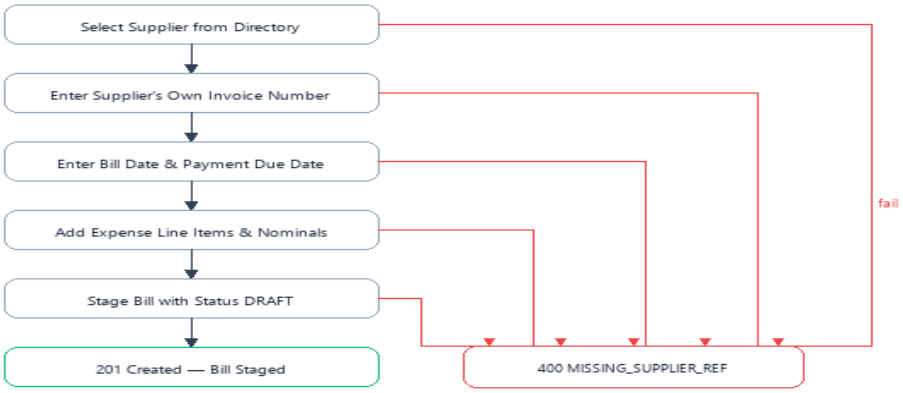


Figure 060 — Supplier Bill Entry with Invoice Reference Flow Architecture

How the System Works: 1. Bookkeeper opens Purchases -> Bills -> '+ New Bill'. ➔ 2. Selects the supplier and types the supplier's own printed invoice number (e.g., 'INV-8891'). ➔ 3. Enters the bill date and payment due date. ➔ 4. Adds item lines, assigning nominal codes (e.g., '7100 - Travel' or '7200 - Office Supplies'). ➔ 5. The bill is saved in 'DRAFT' status ready for review.
Outcome: Bills capture the exact vendor invoice reference required for payment reconciliation. | **Guardrail:** Leaving the supplier invoice reference blank displays: 'Please enter the supplier's invoice number.'

UC-061: Duplicate Supplier Bill Number Interception

Module: Fraud & Error Prevention | **Actor:** System Posting Engine | **Rejection:** 409 DUPLICATE_SUPPLIER_BILL

Business Goal: Prevent paying the same supplier invoice twice.

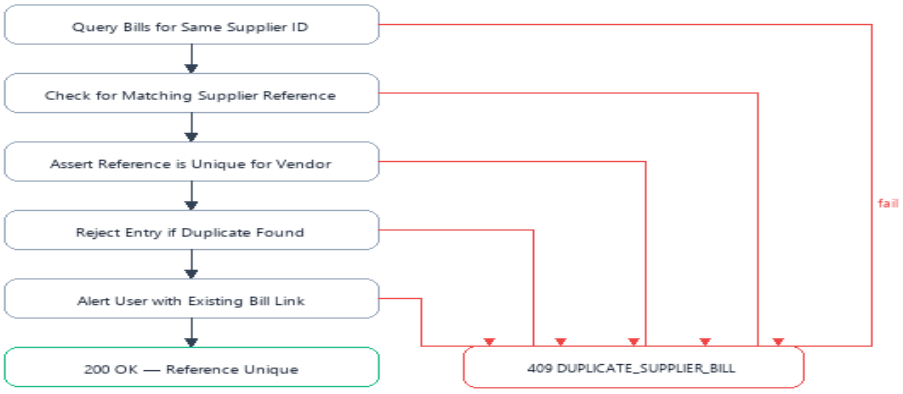


Figure 061 — Duplicate Supplier Bill Number Interception Flow Architecture

How the System Works: 1. A bookkeeper accidentally attempts to enter a bill with supplier invoice number 'INV-8891' a second time. ➔ 2. Blynt queries all existing bills for that specific supplier. ➔ 3. The system detects an existing bill already has reference 'INV-8891'. ➔ 4. The save operation is blocked immediately. ➔ 5. The screen displays a warning with a clickable link to view the existing bill.
Outcome: Businesses are protected from accidental double-payments to suppliers. | **Guardrail:** The screen alerts: 'A bill with reference INV-8891 already exists for this supplier.'

UC-062: Input VAT Recovery Classification Rule

Module: Tax Accounting | **Actor:** Bookkeeper | **Rejection:** 400 INVALID_INPUT_VAT

Business Goal: Reclaim allowable VAT on business purchases on the quarterly VAT return.

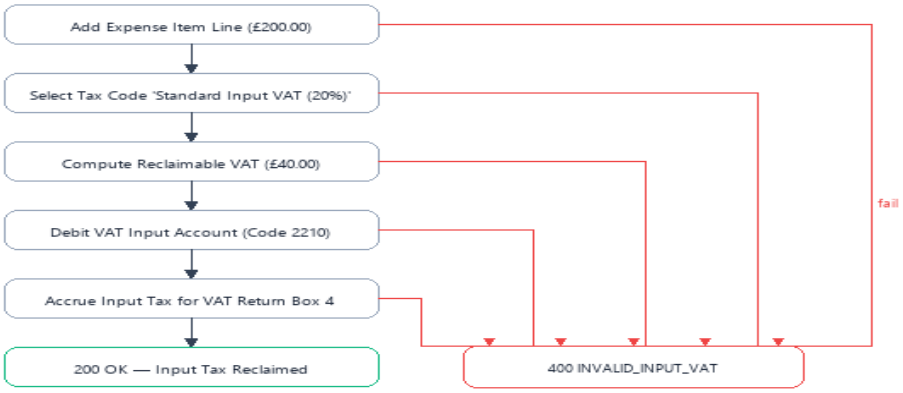


Figure 062 — Input VAT Recovery Classification Rule Flow Architecture

How the System Works: 1. Bookkeeper enters a qualifying business expense bill: Net £500.00 + 20% VAT £100.00 = Total £600.00. ➔ 2. Selects tax rate 'Standard Input VAT (20%)'. ➔ 3. Blynt routes the £100.00 tax to Input Tax nominal Code 2210. ➔ 4. The £100.00 is tagged to report in VAT Return Box 4 (VAT Reclaimed on Purchases). ➔ 5. When the VAT return is filed, this £100.00 reduces the net tax owed to HMRC.
Outcome: The business reclaims all legitimate input VAT, maximizing cash flow. | **Guardrail:** Incorrect tax calculations are flagged for review before posting.

UC-063: Non-Deductible Business Expense Handling

Module: Tax Accounting | **Actor:** Bookkeeper / Accountant | **Rejection:** 400 TAX_TAG_INVALID

Business Goal: Correctly categorize client entertainment so VAT is not illegally reclaimed.

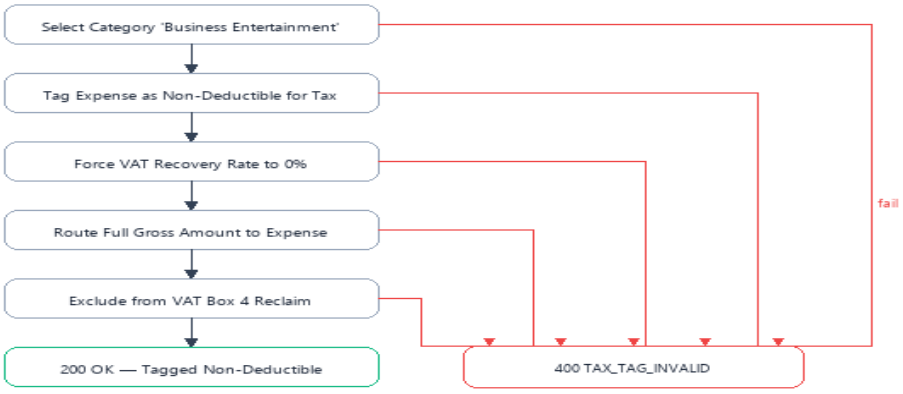


Figure 063 — Non-Deductible Business Expense Handling Flow Architecture

How the System Works: 1. Bookkeeper records a bill for a client dinner (£120.00). ➔ 2. Selects nominal Code 7500 ('Business Entertainment'). ➔ 3. Blynt recognizes that under UK HMRC tax law, client entertainment is NOT tax-deductible for VAT. ➔ 4. The system forces the VAT recovery rate to 0% (£0.00 tax reclaim). ➔ 5. The full £120.00 gross amount is debited to the expense nominal.
Outcome: The business is protected from HMRC tax penalties during compliance audits. | **Guardrail:** Attempting to claim input VAT on client entertainment triggers an automatic tax correction prompt.

UC-064: Accounts Payable General Ledger Posting

Module: Ledger Engine | **Actor:** Bookkeeper / Finance Manager | **Rejection:** 422 AP_POSTING_FAILED

Business Goal: Approve a supplier bill and record the money owed in company accounts.

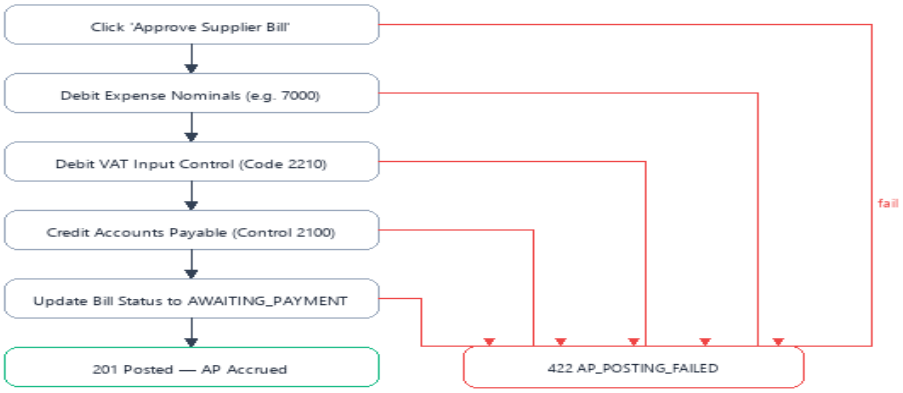


Figure 064 — Accounts Payable General Ledger Posting Flow Architecture

How the System Works: 1. User approves a £600.00 supplier bill (£500 net + £100 VAT). ➔ 2. Blynt atomically posts the General Ledger journal: ➔ 3. - DEBIT Code 7200 (Office Supplies Expense) £500.00 ➔ 4. - DEBIT Code 2210 (VAT Input Tax) £100.00 ➔ 5. - CREDIT Code 2100 (Accounts Payable) £600.00 ➔ 6. The bill status updates to 'AWAITING_PAYMENT'.
Outcome: Company liabilities and expenses are recorded accurately in real time. | **Guardrail:** Posting is blocked if expense nominal codes are inactive or missing.

UC-065: Supplier Outgoing Payment Allocation

Module: Purchases & AP | **Actor:** Bookkeeper | **Rejection:** 400 OVERPAYMENT_UNSUPPORTED

Business Goal: Record paying a supplier from the company bank account.

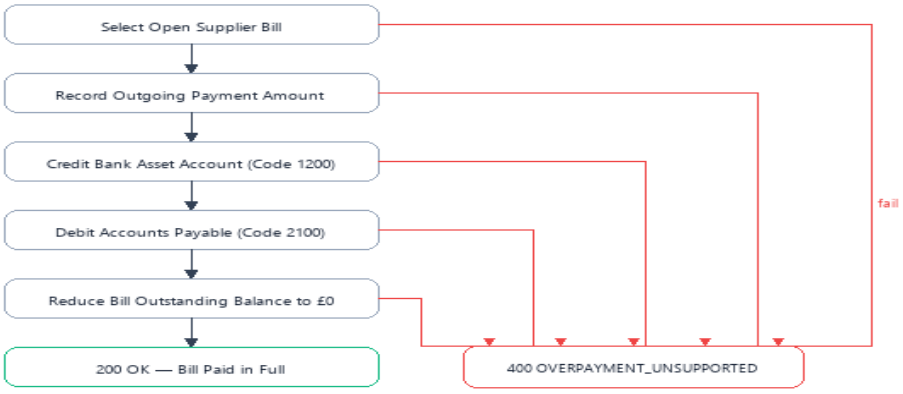


Figure 065 — Supplier Outgoing Payment Allocation Flow Architecture

How the System Works: 1. User opens the approved bill and clicks 'Record Payment'. ➔ 2. Enters payment date, amount (£600.00), and selects Bank Account (Code 1200). ➔ 3. Blynt posts: DEBIT Accounts Payable (2100) £600.00, CREDIT Bank (1200) £600.00. ➔ 4. The bill status flips to 'PAID'. ➔ 5. The supplier's outstanding balance drops to £0.00.
Outcome: Bank cash and Accounts Payable balances stay perfectly synchronized. | **Guardrail:** Attempting to pay more than the bill total displays: 'Payment amount cannot exceed outstanding bill balance.'

UC-066: Supplier Partial Payment Allocation

Module: Purchases & AP | **Actor:** Bookkeeper | **Rejection:** 200 PARTIAL_PAYMENT_RECORDED

Business Goal: Pay an installment on a supplier bill and track the remaining balance.

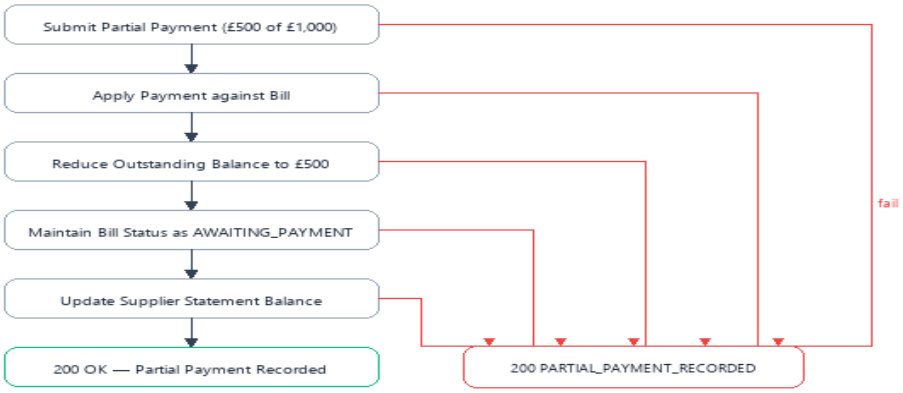


Figure 066 — Supplier Partial Payment Allocation Flow Architecture

How the System Works: 1. User records a partial payment of £200.00 against a £600.00 bill. ➔ 2. Blynt allocates the £200.00 to the bill. ➔ 3. The bill status remains 'AWAITING_PAYMENT', displaying 'Amount Due: £400.00'. ➔ 4. The General Ledger debits Accounts Payable by £200.00. ➔ 5. The supplier's statement shows the initial bill, the payment, and the remaining £400.00 owed.
Outcome: Installment payments are tracked cleanly without losing sight of outstanding debt. | **Guardrail:** The partial payment is confirmed and updated immediately.

UC-067: Supplier Credit Note Offset Application

Module: Purchases & AP | **Actor:** Bookkeeper | **Rejection:** 400 CREDIT_EXCEEDS_BILL

Business Goal: Apply a refund or credit from a supplier against an unpaid bill.

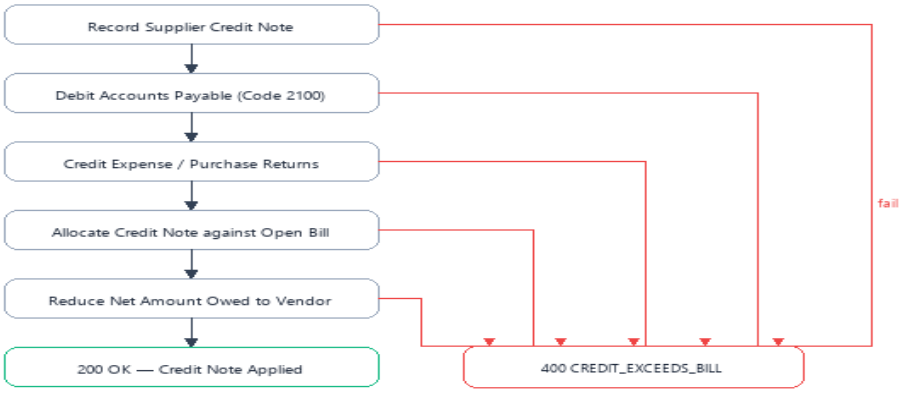


Figure 067 — Supplier Credit Note Offset Application Flow Architecture

How the System Works: 1. A supplier issues a £100.00 credit note for returned goods. ➔ 2. Bookkeeper records the credit note in Purchases -> Credit Notes. ➔ 3. Clicks 'Apply to Open Bill' and selects the £400.00 outstanding bill. ➔ 4. Blynt debits Accounts Payable by £100.00, reducing the bill amount due to £300.00. ➔ 5. Both documents update synchronously.
Outcome: Credit notes reduce amounts owed without needing physical cash refunds. | **Guardrail:** Attempting to apply a credit note larger than the bill displays: 'Credit note exceeds bill amount.'

UC-068: Supplier Remittance Advice Generation

Module: Document Generation | **Actor:** Bookkeeper | **Rejection:** 200 REMITTANCE_DISPATCHED

Business Goal: Send the supplier clear documentation of which bills were paid.

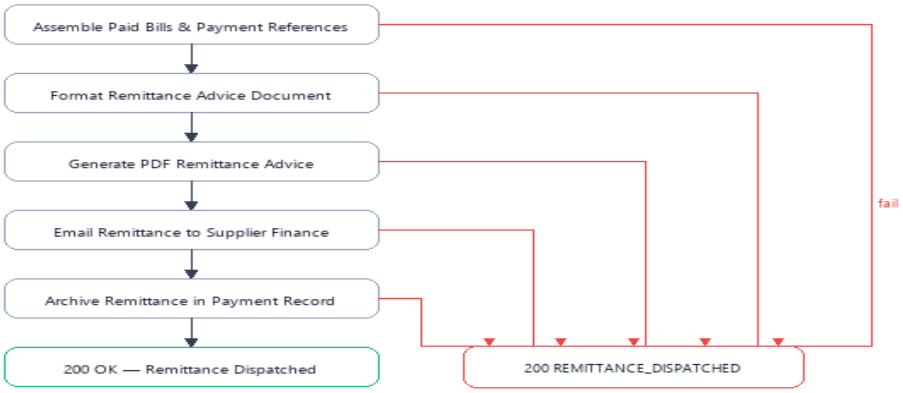


Figure 068 — Supplier Remittance Advice Generation Flow Architecture

How the System Works: 1. User executes a batch payment to a supplier settling multiple bills. ➔ 2. User clicks 'Generate Remittance Advice'. ➔ 3. Blynt compiles a professional A4 PDF listing each invoice reference, original amount, and amount paid. ➔ 4. The remittance advice includes the payment date and bank transfer reference. ➔ 5. The PDF can be emailed directly to the supplier's finance department in one click.
Outcome: Suppliers can allocate payments quickly, preventing mistaken chase calls. | **Guardrail:** The remittance advice is created and archived in the payment record.

UC-069: Mobile Photo Receipt Upload & Normalization

Module: AI Intelligence | **Actor:** Employee / Director on the road | **Rejection:** 400 UNSUPPORTED_IMAGE_FORMAT

Business Goal: Capture paper receipts using a smartphone camera.

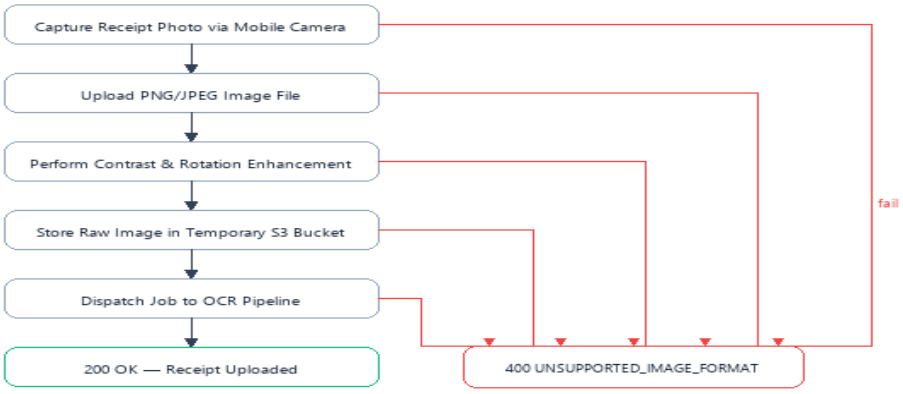


Figure 069 — Mobile Photo Receipt Upload & Normalization Flow Architecture

How the System Works: 1. User takes a photo of a restaurant receipt on their phone and uploads it to Blynt Expenses. ➔ 2. Blynt inspects the file, ensuring it is a valid JPEG or PNG. ➔ 3. An automated image pre-processor corrects rotation, enhances contrast, and flattens shadows. ➔ 4. The normalized image is stored temporarily in secure cloud storage. ➔ 5. The document is queued for OCR text recognition.
Outcome: Paper receipts are captured in seconds before they get lost or faded. | **Guardrail:** Uploading a corrupted file shows: 'Could not process image. Please upload a clear photo.'

UC-070: Multi-Page PDF Supplier Bill Parsing

Module: AI Intelligence | **Actor:** Bookkeeper | **Rejection:** 400 CORRUPTED_PDF

Business Goal: Process multi-page digital invoices received via email.

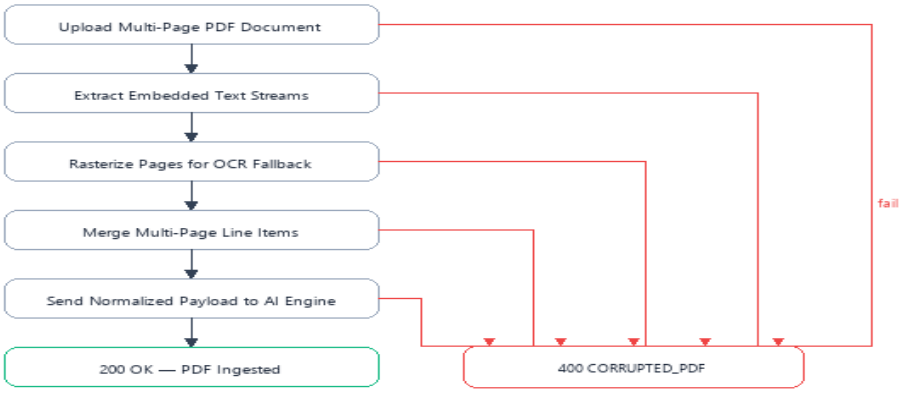


Figure 070 — Multi-Page PDF Supplier Bill Parsing Flow Architecture

How the System Works: 1. User drags a 3-page supplier PDF into the Blynt upload box. ➔ 2. The document parser extracts embedded digital text streams. ➔ 3. If text is scanned, OCR fallback processes the high-resolution pages. ➔ 4. The system combines line items across all 3 pages into a single digital bill payload. ➔ 5. The extracted data is presented for user review.
Outcome: Long, complex supplier bills are digitized in seconds without manual typing. | **Guardrail:** Password-protected or corrupted PDFs display: 'This PDF is encrypted. Please provide an unlocked file.'

UC-071: AI Vendor Identification & Confidence Scoring

Module: AI Intelligence | **Actor:** Blynt AI Engine | **Rejection:** 422 VENDOR_UNRECOGNIZED

Business Goal: Identify the merchant name accurately using machine learning.

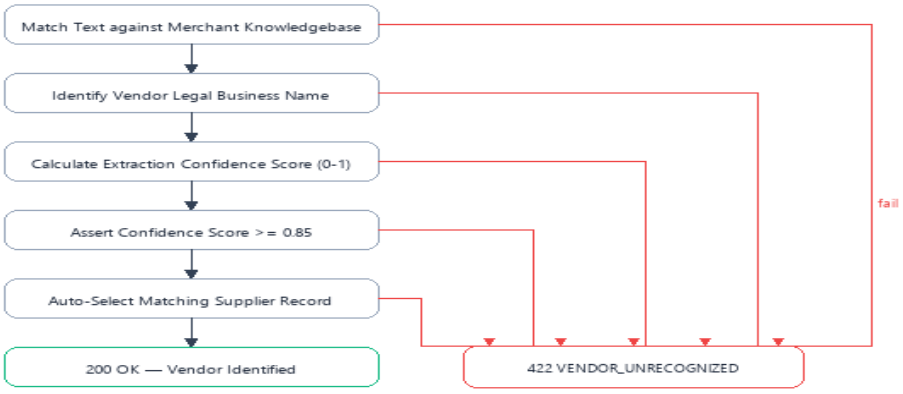


Figure 071 — AI Vendor Identification & Confidence Scoring Flow Architecture

How the System Works: 1. Document AI scans the top of the receipt for merchant branding (e.g., 'Pret A Manger'). ➔ 2. Matches the text against a global merchant directory. ➔ 3. Calculates an extraction confidence score between 0.00 and 1.00. ➔ 4. If confidence is >= 0.85, Blynt automatically matches or creates the supplier record. ➔ 5. If confidence is low, the field is highlighted in yellow for human confirmation.
Outcome: Merchant names are filled in automatically with high accuracy. | **Guardrail:** Unrecognizable merchant text prompts the user to type the name manually.

UC-072: AI Date & Amount Precision Parsing

Module: AI Intelligence | **Actor:** Blynt AI Engine | **Rejection:** 422 DATE_PARSE_FAILED

Business Goal: Extract dates and monetary totals from scanned documents.

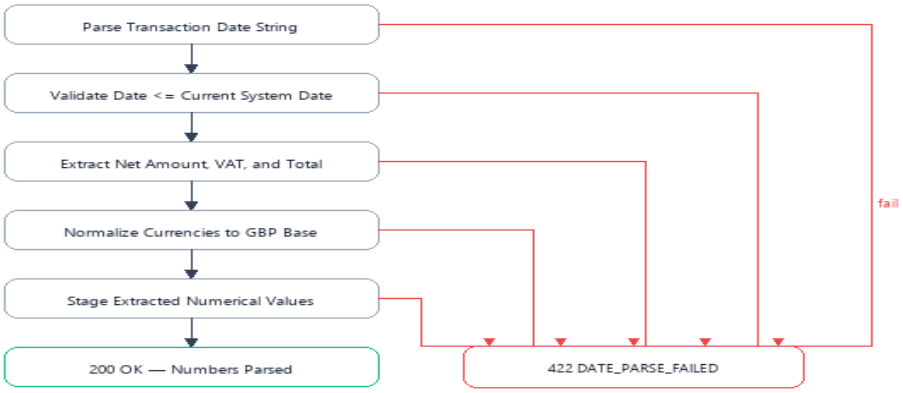


Figure 072 — AI Date & Amount Precision Parsing Flow Architecture

How the System Works: 1. The AI engine identifies numerical strings representing currency amounts and dates. ➔ 2. Extracts Transaction Date, Net Amount, VAT Amount, and Total Amount. ➔ 3. Validates that the extracted date is valid and is not in the future. ➔ 4. Converts foreign currency symbols (£, $, €) into standardized monetary values. ➔ 5. Pre-fills the expense form fields on screen.
Outcome: Typing numbers manually is eliminated, cutting expense processing time by 80%. | **Guardrail:** Dates that cannot be parsed clearly prompt the user to pick a date from the calendar.

UC-073: AI Arithmetic Balancing Guardrail Rule

Module: AI Safety | **Actor:** Blynt Safety Engine | **Rejection:** 422 AI_MATH_MISMATCH

Business Goal: Stop AI hallucinations: ensure extracted line items add up to the total.

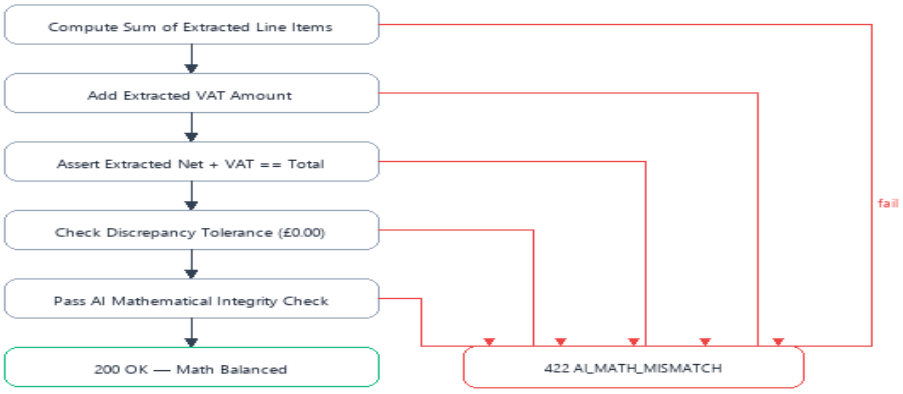


Figure 073 — AI Arithmetic Balancing Guardrail Rule Flow Architecture

How the System Works: 1. The AI engine finishes extracting Net, VAT, and Total amounts. ➔ 2. Before showing suggestions, Blynt's safety guardrail executes: 'Assert Net + VAT == Total'. ➔ 3. If the extracted items do not add up to the total (e.g., OCR misread a 3 as an 8), the AI flag triggers. ➔ 4. The system refuses to auto-fill hallucinated numbers. ➔ 5. The side-by-side verification screen highlights the discrepancy in red for human correction.
Outcome: AI mistakes are caught before they ever reach company books. | **Guardrail:** Discrepancies are highlighted: 'Extracted amounts do not match receipt total. Please verify.'

UC-074: Low Confidence Human-in-the-Loop Intercept

Module: AI Safety | **Actor:** Blynt Safety Engine | **Rejection:** 422 HUMAN_REVIEW_ENFORCED

Business Goal: Enforce human review whenever receipt text is blurry or ambiguous.

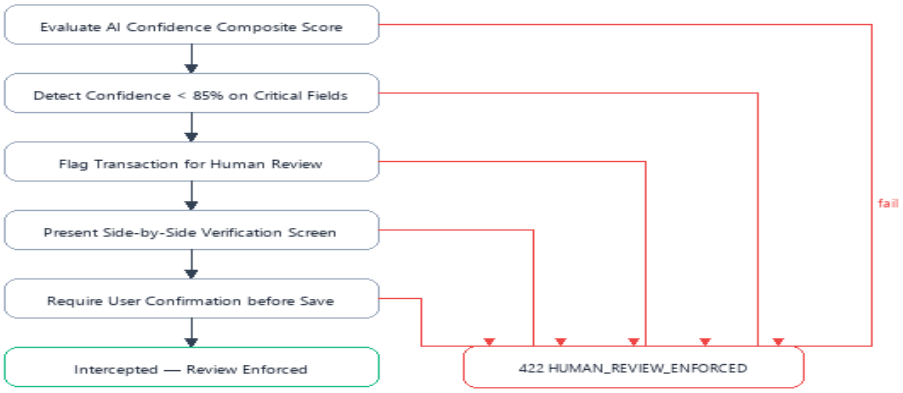


Figure 074 — Low Confidence Human-in-the-Loop Intercept Flow Architecture

How the System Works: 1. A user uploads a crumpled, faded receipt. ➔ 2. The AI composite confidence score falls to 68% (below the strict 85% safety threshold). ➔ 3. Blynt intercepts automated posting. ➔ 4. The screen displays the receipt image on the left and input fields on the right. ➔ 5. The user must review and click 'Confirm & Save' before the expense is committed.
Outcome: Blynt combines AI speed with human oversight: zero unchecked entries reach the ledger. | **Guardrail:** Unclear receipts are held safely in the review queue until approved.

UC-075: Smart Expense Category Recommendation

Module: Machine Learning | **Actor:** Blynt AI Engine | **Rejection:** 200 CATEGORY_SUGGESTED

Business Goal: Automatically categorize expenses based on past company habits.

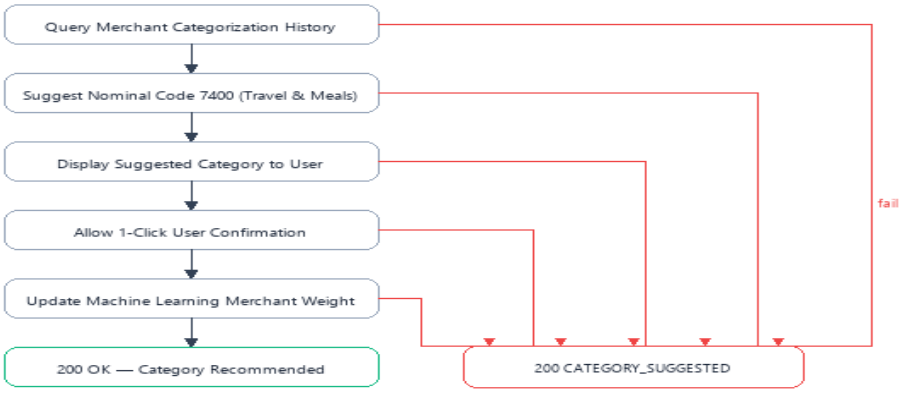


Figure 075 — Smart Expense Category Recommendation Flow Architecture

How the System Works: 1. The AI recognizes merchant 'Shell Petrol Station'. ➔ 2. The recommendation engine checks past company categorizations for this merchant. ➔ 3. Identifies that Shell was categorized as '7300 - Motor Expenses' 100% of the time. ➔ 4. Pre-selects Code 7300 in the dropdown with a green sparkle icon: 'Recommended'. ➔ 5. User can accept with one click or choose another category.
Outcome: Bookkeeping categorization becomes instant and consistent across all staff. | **Guardrail:** User can override the recommendation at any time.

UC-076: Permanent Cloud Storage Image Archiving

Module: Statutory Compliance | **Actor:** Storage Service | **Rejection:** 500 S3_UPLOAD_FAILED

Business Goal: Attach receipt photos permanently to accounting records for tax audits.

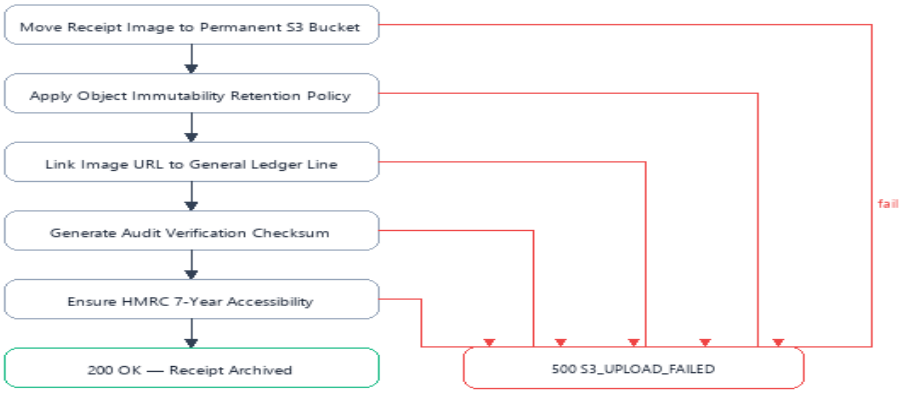


Figure 076 — Permanent Cloud Storage Image Archiving Flow Architecture

How the System Works: 1. When an expense is saved, the receipt photo is moved to encrypted, permanent cloud storage. ➔ 2. An immutable storage URL is saved directly inside the General Ledger transaction record. ➔ 3. A SHA-256 cryptographic checksum of the image is computed and stored. ➔ 4. If a tax auditor inspects the expense 5 years later, clicking 'View Receipt' opens the original photo. ➔ 5. The file cannot be overwritten or deleted.
Outcome: Businesses meet HMRC 7-year digital receipt storage requirements with zero paper folders. | **Guardrail:** Storage upload failure aborts the save to prevent receipts from being lost.

UC-077: Business Bank Account Nominal Account Binding

Module: Banking Setup | **Actor:** Accountant | **Rejection:** 400 INVALID_BANK_NOMINAL

Business Goal: Connect a business bank account directly to the general ledger.

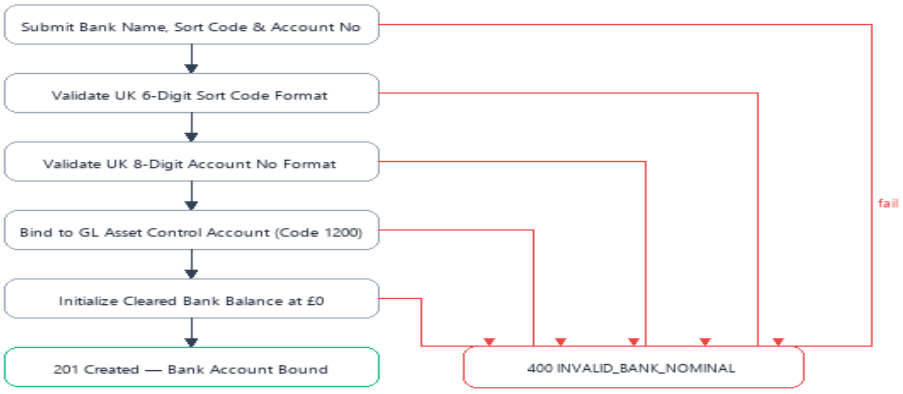


Figure 077 — Business Bank Account Nominal Account Binding Flow Architecture

How the System Works: 1. User opens Banking -> Bank Accounts -> '+ Add Bank Account'. ➔ 2. Enters bank name (e.g., 'Barclays Current Account'), Sort Code (20-04-01), and Account Number. ➔ 3. Blynt binds the physical bank account to General Ledger Code 1200 (Bank Asset Account). ➔ 4. The system initializes the cleared bank balance at £0.00. ➔ 5. The account is ready to receive electronic statements and track cash balances.
Outcome: Bank cash movements are tracked inside the General Ledger with double-entry integrity. | **Guardrail:** Entering an invalid sort code format shows: 'Please enter a valid 6-digit UK sort code.'

UC-078: Bank Statement CSV Feed Parsing

Module: Banking Feeds | **Actor:** Bookkeeper | **Rejection:** 400 MALFORMED_CSV_HEADER

Business Goal: Import downloaded bank statements without manual line typing.

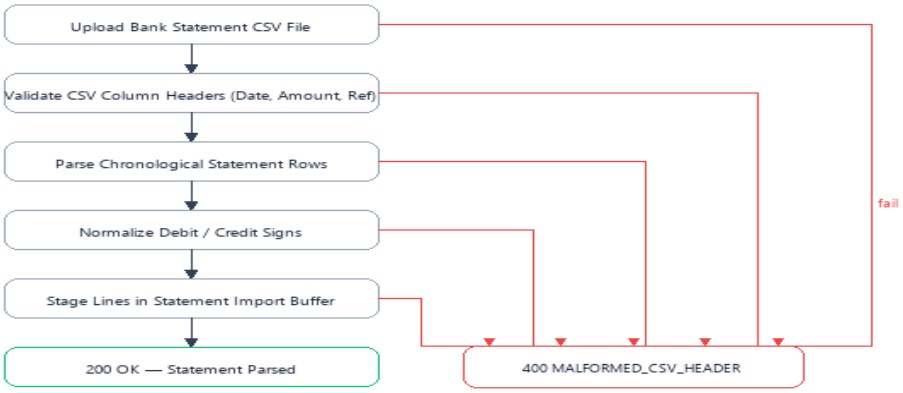


Figure 078 — Bank Statement CSV Feed Parsing Flow Architecture

How the System Works: 1. User downloads a monthly CSV statement from their online bank and uploads it to Blynt. ➔ 2. Blynt's statement parser inspects column headers (Date, Description, Amount, Balance). ➔ 3. Normalizes money formats, handling debit/credit columns or positive/negative amounts. ➔ 4. Stages all transactions into the bank reconciliation staging buffer. ➔ 5. Displays the transactions in the bank feed inbox ready for matching.
Outcome: Hundreds of bank transactions are imported in seconds without typing errors. | **Guardrail:** Uploading a corrupted CSV file shows: 'Could not read CSV headers. Please verify file format.'

UC-079: Bank Statement Duplicate Line Hash Intercept

Module: Banking Feeds | **Actor:** Banking Safety Engine | **Rejection:** 409 DUPLICATE_BANK_FEED_LINE

Business Goal: Prevent importing the same bank transaction twice when statements overlap.

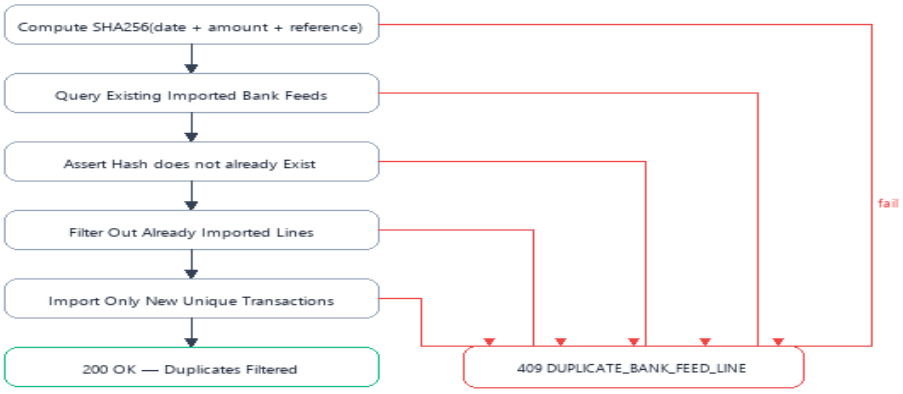


Figure 079 — Bank Statement Duplicate Line Hash Intercept Flow Architecture

How the System Works: 1. User uploads a statement covering the 1st to the 31st, having previously uploaded the 1st to the 15th. ➔ 2. For every line, Blynt computes a unique fingerprint: SHA-256(Date + Amount + Description). ➔ 3. Queries the database for existing fingerprints on this bank account. ➔ 4. The system detects that lines from the 1st to the 15th were already imported. ➔ 5. Duplicates are silently filtered out; only the brand-new lines from the 16th to 31st are imported.
Outcome: Bank balances never double-count transactions due to overlapping statement imports. | **Guardrail:** The system confirms: '32 new transactions imported. 28 overlapping lines were safely skipped.'

UC-080: Automated Rule-Based Invoice Matching

Module: Bank Reconciliation | **Actor:** Reconciliation Engine | **Rejection:** 200 AUTO_MATCHED

Business Goal: Automatically match customer payments with unpaid invoices.

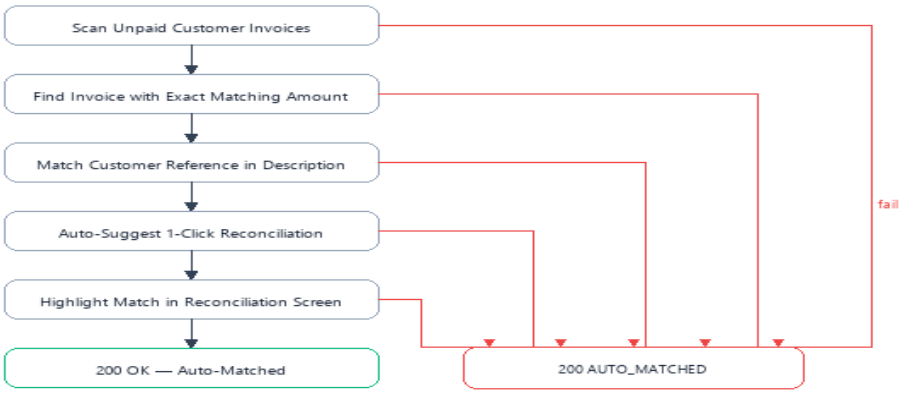


Figure 080 — Automated Rule-Based Invoice Matching Flow Architecture

How the System Works: 1. A bank transaction shows a £1,200.00 deposit with description 'Acme UK Ltd INV-1042'. ➔ 2. Blynt's matching engine scans open customer invoices. ➔ 3. Finds Invoice #1042 for Acme UK Ltd with exact amount £1,200.00. ➔ 4. The screen presents a green 'Match Found' card: 'Match deposit to Invoice #1042?'. ➔ 5. User clicks 'OK' to reconcile in one second.
Outcome: Bank reconciliation time is cut by 90% through automated matching. | **Guardrail:** Matches are presented clearly for human confirmation before clearing.

UC-081: Manual Multi-Transaction Bank Split Reconciliation

Module: Bank Reconciliation | **Actor:** Accountant | **Rejection:** 400 SPLIT_MISMATCH

Business Goal: Split a single lump-sum deposit across multiple customer invoices.

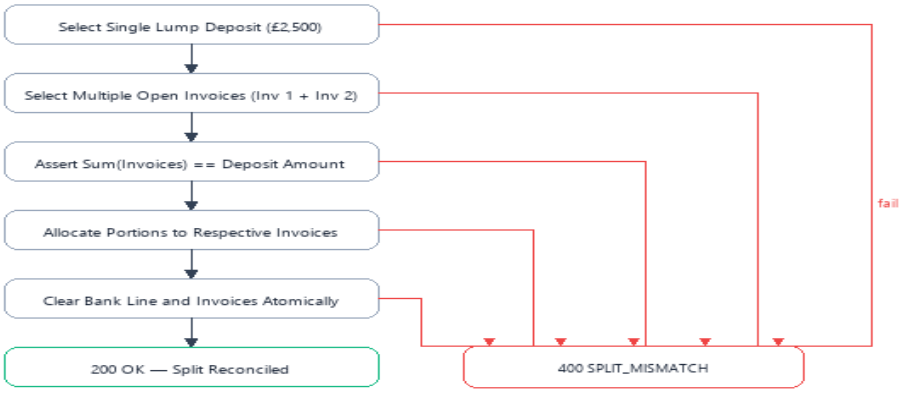


Figure 081 — Manual Multi-Transaction Bank Split Reconciliation Flow Architecture

How the System Works: 1. A customer pays £2,500.00 in a single bank transfer to settle two invoices (Inv A £1,500 and Inv B £1,000). ➔ 2. Accountant selects the £2,500.00 bank statement line and clicks 'Find & Match'. ➔ 3. Selects Invoice A and Invoice B from the open invoices list. ➔ 4. Blynt verifies: 'Invoice A (£1,500) + Invoice B (£1,000) == Deposit (£2,500)'. ➔ 5. Clicks 'Reconcile'—both invoices flip to PAID and the bank line is cleared.
Outcome: Lump-sum customer payments are reconciled accurately across multiple accounts. | **Guardrail:** If the selected invoices do not equal the deposit, reconciliation is blocked until balanced.

UC-082: Unreconciled Bank Balance vs GL Equilibrium

Module: Ledger Integrity | **Actor:** Accountant | **Rejection:** 200 BANK_EQUILIBRIUM_OK

Business Goal: Verify that company accounting books match actual bank cash exactly.

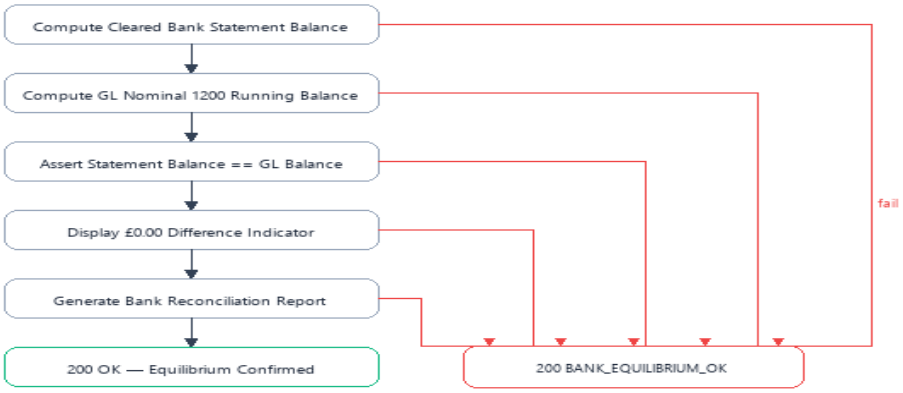


Figure 082 — Unreconciled Bank Balance vs GL Equilibrium Flow Architecture

How the System Works: 1. Accountant opens the Bank Reconciliation Summary report at month end. ➔ 2. Blynt calculates: Actual Statement Ending Balance vs Cleared General Ledger Balance (Code 1200). ➔ 3. Displays outstanding uncleared cheques or deposits in transit. ➔ 4. Asserts that the adjusted bank balance matches the ledger balance down to the penny. ➔ 5. Displays a green badge: 'Difference: £0.00 — Fully Reconciled'.
Outcome: Business owners have 100% confidence that their financial reports reflect real money in the bank. | **Guardrail:** Any discrepancy is highlighted in red, allowing bookkeepers to trace missing lines immediately.

UC-083: Reconciliation Clearance Locking Rule

Module: Audit Safety | **Actor:** Accountant | **Rejection:** 403 CLEARED_TRANSACTION_LOCKED

Business Goal: Lock reconciled bank transactions to protect past accounting periods.



Figure 083 — Reconciliation Clearance Locking Rule Flow Architecture

How the System Works: 1. Once a bank statement line is reconciled, Blynt sets its status to 'RECONCILED'. ➔ 2. The transaction is locked against accidental deletion or unmatching. ➔ 3. If a user needs to unmatch a cleared transaction, the system requires accountant confirmation. ➔ 4. The unmatching event and reason are permanently logged in the audit trail. ➔ 5. Past reconciled bank statements remain permanently stable.
Outcome: Reconciled months cannot be accidentally broken by daily operational staff. | **Guardrail:** Attempting to casually delete cleared lines displays: 'This transaction is reconciled and locked.'

UC-084: Realised Foreign Currency Exchange Gain/Loss

Module: Multi-Currency | **Actor:** Ledger Engine | **Rejection:** 200 FX_GAIN_LOSS_POSTED

Business Goal: Automatically calculate currency profit/loss when foreign invoices are paid.

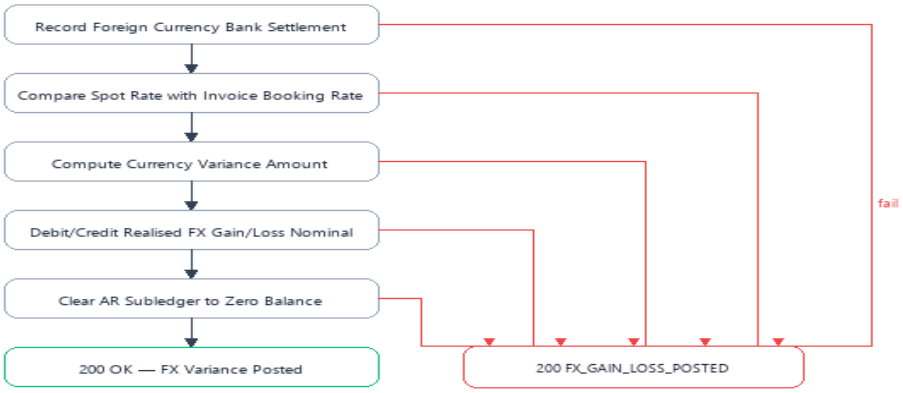


Figure 084 — Realised Foreign Currency Exchange Gain/Loss Flow Architecture

How the System Works: 1. A US client is billed $1,000.00 when the exchange rate is 1.25 (booked at £800.00 GBP). ➔ 2. When the client pays, the exchange rate has moved to 1.20 (the bank receives £833.33 GBP). ➔ 3. Blynt clears the $1,000.00 Accounts Receivable balance (£800.00). ➔ 4. Records cash received in Bank: £833.33. ➔ 5. Automatically posts the £33.33 difference to Nominal Code 4900 ('Realised FX Gain').
Outcome: Currency gains and losses are accounted for automatically under UK GAAP. | **Guardrail:** Exchange variances reconcile seamlessly to the Profit & Loss statement.

UC-085: Monthly Accounting Period Calendar Partitioning

Module: Period Governance | **Actor:** Certified Accountant | **Rejection:** 400 PERIOD_OVERLAP

Business Goal: Establish the 12 calendar partitions for monthly management accounts.

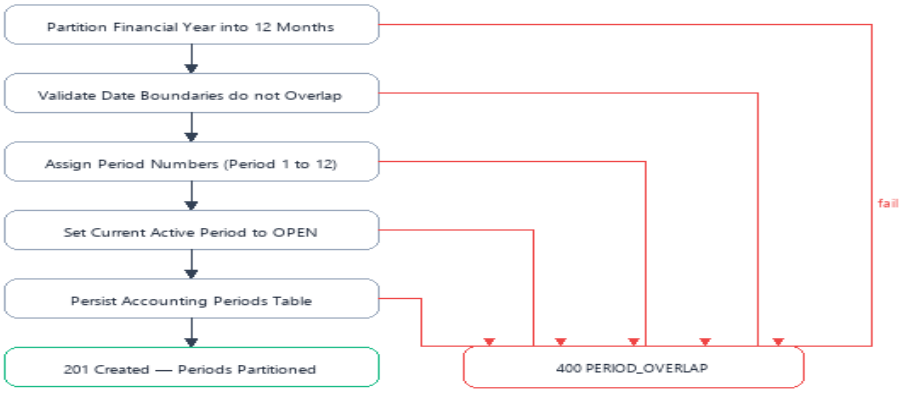


Figure 085 — Monthly Accounting Period Calendar Partitioning Flow Architecture

How the System Works: 1. At the start of the financial year, Blynt partitions the fiscal calendar into 12 monthly periods. ➔ 2. Period 1 (April), Period 2 (May) ... Period 12 (March). ➔ 3. The system verifies that start and end dates cover every day of the year with zero gaps. ➔ 4. Period 1 is set to 'OPEN', and future periods are staged. ➔ 5. Transactions are automatically tagged with their matching accounting period upon entry.
Outcome: Management accounts can be compared month-by-month for financial performance analysis. | **Guardrail:** Overlapping dates are blocked by database integrity constraints.

UC-086: Interim Monthly Accounting Period Close

Module: Period Governance | **Actor:** Certified Accountant | **Rejection:** 403 UNPOSTED_DRAFTS_IN_PERIOD

Business Goal: Lock a completed month so employees cannot alter finalized numbers.

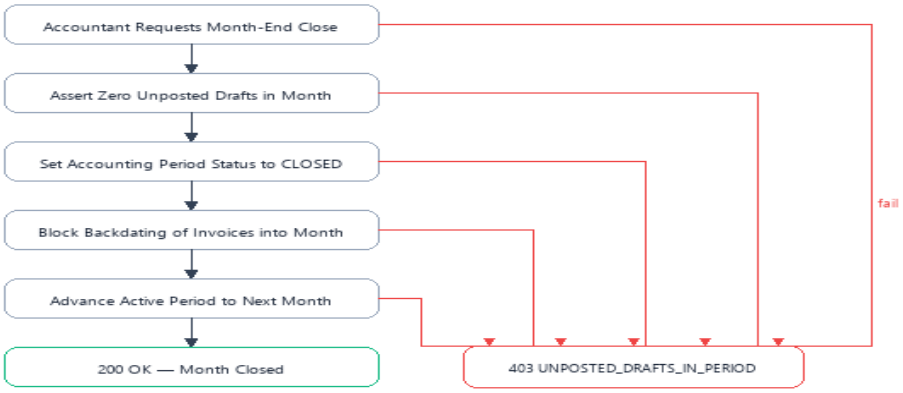


Figure 086 — Interim Monthly Accounting Period Close Flow Architecture

How the System Works: 1. At the end of April, the accountant finishes all bank reconciliations and adjustments. ➔ 2. Accountant navigates to Settings -> Financial Periods and clicks 'Close Period 1 (April)'. ➔ 3. Blynt verifies that there are zero unposted draft journals or invoices dated in April. ➔ 4. The period status updates to 'CLOSED'. ➔ 5. Any future attempt to backdate an invoice into April is blocked.
Outcome: Monthly numbers reported to directors or investors remain permanently stable. | **Guardrail:** Attempting to close a month with unposted drafts displays: 'Please post or discard all April drafts first.'

UC-087: Financial Year-End Final Trial Balance Check

Module: Year-End Closing | **Actor:** Certified Accountant | **Rejection:** 422 LEDGER_NOT_IN_BALANCE

Business Goal: Verify complete ledger equilibrium before initiating the annual tax close.

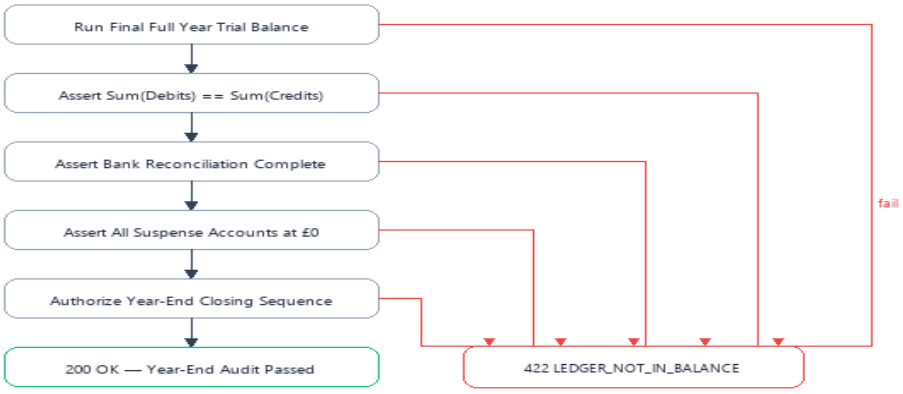


Figure 087 — Financial Year-End Final Trial Balance Check Flow Architecture

How the System Works: 1. Accountant prepares to close Financial Year 2024. ➔ 2. Blynt executes a comprehensive pre-close integrity audit: ➔ 3. 1. Asserts sum(Debits) == sum(Credits) across all accounts. ➔ 4. 2. Asserts all bank accounts are 100% reconciled to the fiscal year end date. ➔ 5. 3. Asserts suspense and clearing accounts hold a balance of exactly £0.00. ➔ 6. If all checks pass, the system authorizes the year-end close sequence.
Outcome: Year-end closing can only occur when the accounts are in perfect, audited balance. | **Guardrail:** If the trial balance is out of equilibrium, the close is blocked with a diagnostic report.

UC-088: Annual Retained Earnings Rollover Engine

Module: Year-End Closing | **Actor:** Accounting Engine | **Rejection:** 500 ROLLOVER_FAILED

Business Goal: Transfer annual profit to equity and reset revenue/expenses for the new year.

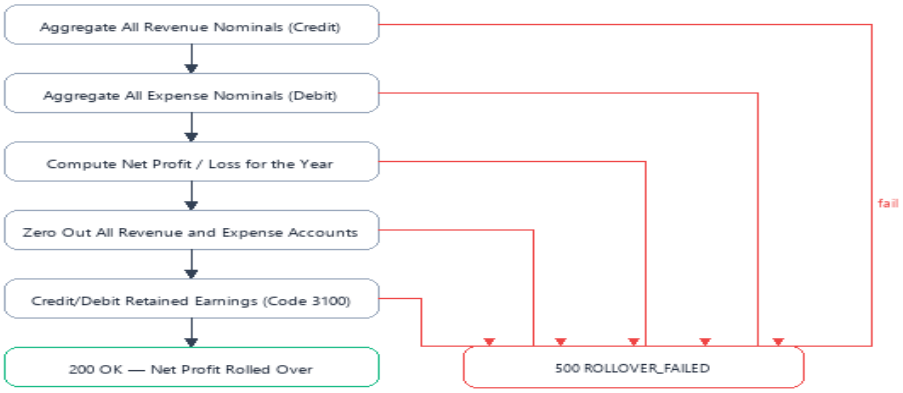


Figure 088 — Annual Retained Earnings Rollover Engine Flow Architecture

How the System Works: 1. Accountant initiates the annual year-end close. ➔ 2. Blynt sums all Revenue nominals (Credit) and all Expense nominals (Debit). ➔ 3. Calculates the net profit for the year (e.g., £54,200.00). ➔ 4. Generates an automated Year-End Closing Journal that resets all Revenue & Expense nominals to £0.00. ➔ 5. Credits the £54,200.00 profit into Equity Code 3100 (Retained Earnings).
Outcome: The new financial year starts with clean £0.00 P&L; accounts, and cumulative profit is preserved in Equity. | **Guardrail:** Any failure during rollover rolls back the transaction safely.

UC-089: Temporal Database Date-Lock Enforcement

Module: Statutory Immutability | **Actor:** PostgreSQL Engine | **Rejection:** 403 PERIOD_TEMPORALLY_LOCKED

Business Goal: Enforce physical database locks preventing backdating into closed tax years.

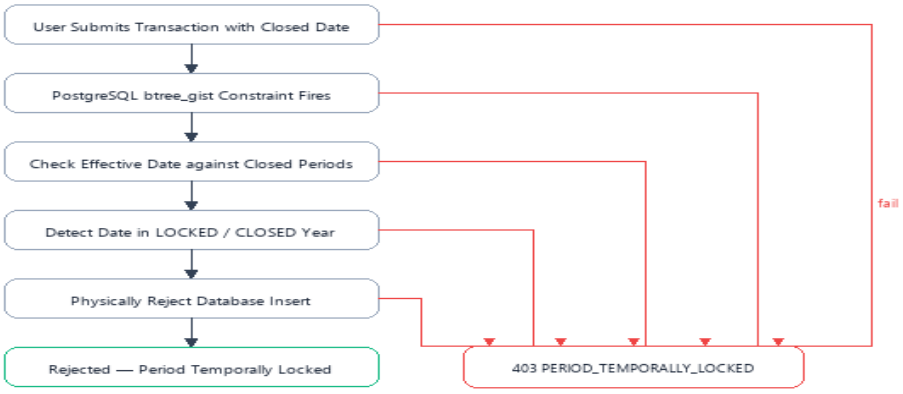


Figure 089 — Temporal Database Date-Lock Enforcement Flow Architecture

How the System Works: 1. A user attempts to create a supplier bill dated 15 November 2023 (inside a CLOSED financial year). ➔ 2. The transaction is sent to PostgreSQL. ➔ 3. A database-level temporal constraint (btree_gist) evaluates the transaction date against closed periods. ➔ 4. The database rejects the INSERT operation with an integrity exception. ➔ 5. The user sees an immediate warning on screen: 'Cannot backdate transactions into a locked financial year.'
Outcome: Tax accounts filed with HMRC cannot be tampered with by any user or software bug. | **Guardrail:** The database physically rejects unauthorized historical modifications.

UC-090: Audited Emergency Period Reopening Workflow

Module: Period Governance | **Actor:** Company Owner Only | **Rejection:** 403 REOPEN_AUDIT_LOG_MANDATORY

Business Goal: Handle rare statutory audit adjustments in closed years with full accountability.



Figure 090 — Audited Emergency Period Reopening Workflow Flow Architecture

How the System Works: 1. A statutory tax auditor requires an adjusting entry in a closed financial year. ➔ 2. The company OWNER navigates to Settings -> Financial Years and requests 'Emergency Unlock'. ➔ 3. Blynt requires two-factor authentication and a mandatory written justification reason. ➔ 4. The system logs the exact timestamp, user ID, IP address, and justification into the immutable audit trail. ➔ 5. The period unlocks for a temporary 24-hour window, after which it automatically re-locks.
Outcome: Legitimate auditor adjustments can be made without compromising corporate governance. | **Guardrail:** Attempting to reopen without a written justification is blocked: 'Justification reason is required.'

UC-091: Trial Balance Report Equilibrium Derivation

Module: Financial Reporting | **Actor:** Accountant / Auditor | **Rejection:** 422 TB_OUT_OF_BALANCE

Business Goal: Generate the master trial balance proving ledger balance across all nominals.

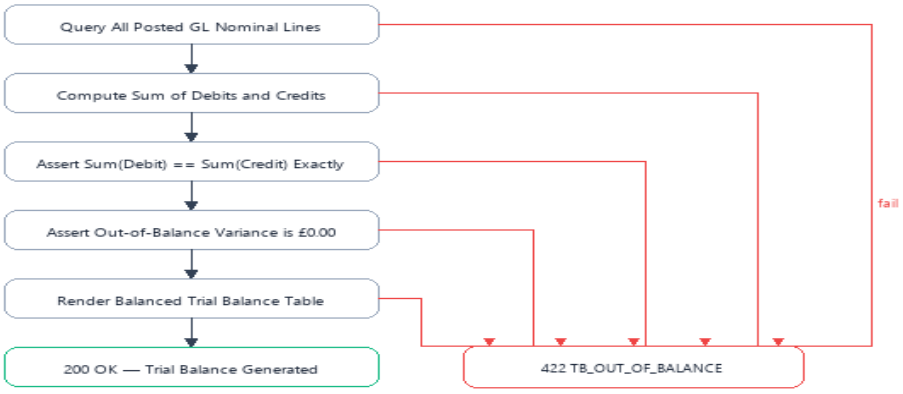


Figure 091 — Trial Balance Report Equilibrium Derivation Flow Architecture

How the System Works: 1. User opens Reports -> Trial Balance and selects a date range. ➔ 2. Blynt aggregates all debit and credit journal lines across every nominal account. ➔ 3. Computes the net balance for each account according to its normal balance type. ➔ 4. Sums all Debit balances and all Credit balances. ➔ 5. Asserts that Total Debits equals Total Credits down to the exact penny (£0.00 variance).
Outcome: Accountants have an instant, reliable snapshot of the entire general ledger for tax filings. | **Guardrail:** Any discrepancy triggers an internal diagnostic flag for audit investigation.

UC-092: Profit & Loss (Income Statement) Derivation

Module: Financial Reporting | **Actor:** Business Owner / CPA | **Rejection:** 200 PNL_GENERATED

Business Goal: Track sales turnover, gross margins, overhead expenses, and net profit.

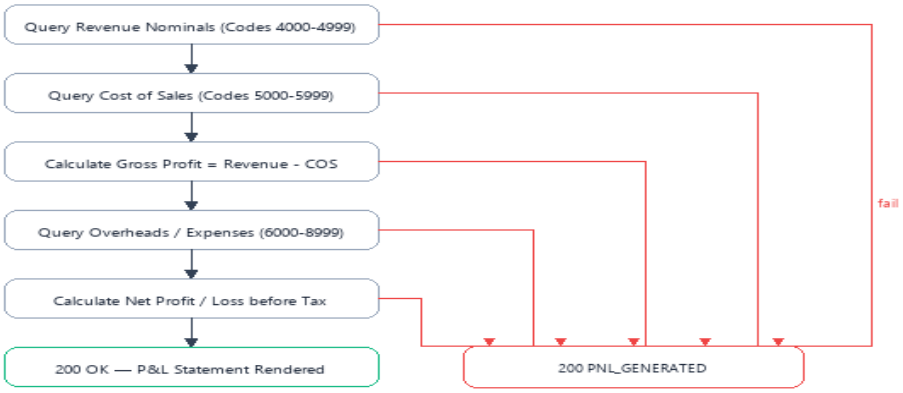


Figure 092 — Profit & Loss (Income Statement) Derivation Flow Architecture

How the System Works: 1. User opens Reports -> Profit & Loss for the current quarter. ➔ 2. Blynt pulls all Revenue nominals (Codes 4000-4999) to compute Operating Turnover. ➔ 3. Subtracts Cost of Sales (Codes 5000-5999) to compute Gross Profit Margin. ➔ 4. Subtracts Operating Expenses (Codes 6000-8999) to compute Net Operating Profit. ➔ 5. Displays the report on screen with 1-click export to PDF or Excel.
Outcome: Business leaders can see exact profitability and expense trends in real time. | **Guardrail:** All figures can be clicked to drill down into the underlying invoices and receipts.

UC-093: Balance Sheet Statement Derivation & Balance

Module: Financial Reporting | **Actor:** Accountant / Lender | **Rejection:** 422 BALANCE_SHEET_UNBALANCED

Business Goal: Verify the company's financial standing: Assets = Liabilities + Equity.

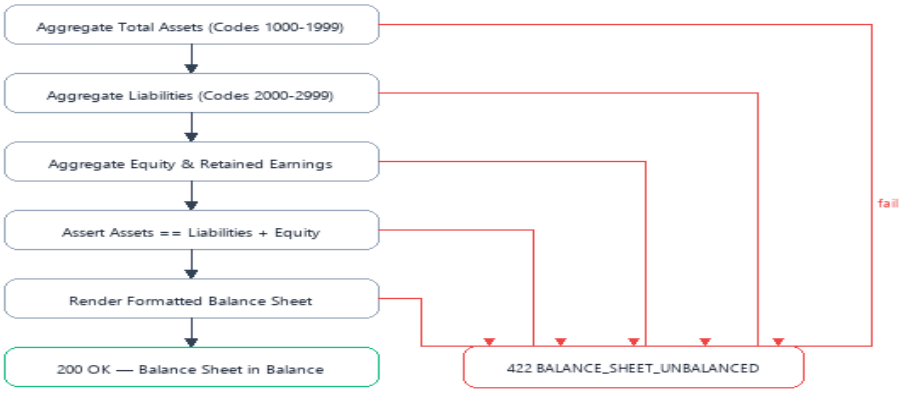


Figure 093 — Balance Sheet Statement Derivation & Balance Flow Architecture

How the System Works: 1. User opens Reports -> Balance Sheet at the close of the month. ➔ 2. Blynt aggregates Current and Fixed Assets (Codes 1000-1999). ➔ 3. Aggregates Current and Long-Term Liabilities (Codes 2000-2999). ➔ 4. Aggregates Capital, Retained Earnings, and current year Net Profit. ➔ 5. Asserts the golden balance equation: Total Assets == Total Liabilities + Total Equity.
Outcome: Provides banks, lenders, and directors with an audited statement of business net worth. | **Guardrail:** An out-of-balance condition is mathematically prevented by double-entry invariants.

UC-094: Quarterly HMRC VAT Return Box 1-9 Derivation

Module: Tax Compliance | **Actor:** Certified Accountant | **Rejection:** 422 VAT_CALCULATION_INVALID

Business Goal: Derive standard UK VAT Return figures directly from ledger lines.

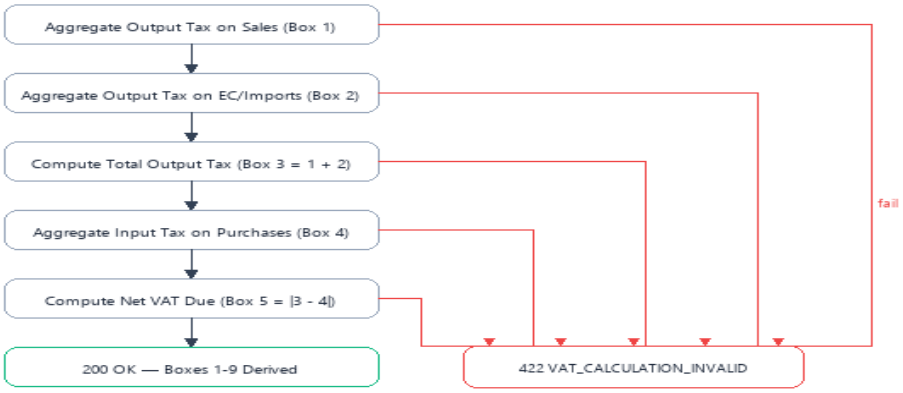


Figure 094 — Quarterly HMRC VAT Return Box 1-9 Derivation Flow Architecture

How the System Works: 1. Accountant selects the VAT quarter (e.g., 01 Jan to 31 Mar). ➔ 2. Blynt aggregates all tax-tagged transactions posted in that period: ➔ 3. - Box 1: VAT due on sales and other outputs ➔ 4. - Box 3: Total VAT due (Box 1 + Box 2) ➔ 5. - Box 4: VAT reclaimed on purchases and inputs ➔ 6. - Box 5: Net VAT to pay to HMRC or reclaim (Box 3 minus Box 4) ➔ 7. - Box 6 & 7: Total value of sales and purchases excluding VAT. ➔ 8. Validates that all box arithmetic reconciles perfectly to General Ledger tax control nominals.
Outcome: Quarterly VAT returns are prepared in seconds with zero manual copy-paste errors. | **Guardrail:** Any mathematical discrepancy halts the return: 'VAT calculation error detected.'

UC-095: Making Tax Digital (MTD) HMRC API Submission

Module: Statutory Compliance | **Actor:** Certified Accountant | **Rejection:** 502 HMRC_GATEWAY_REJECTED

Business Goal: Submit VAT returns directly to HMRC over the government MTD API.

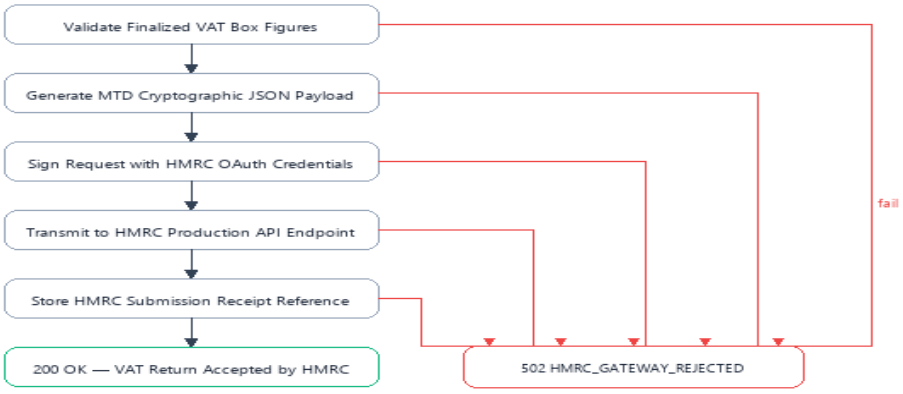


Figure 095 — HMRC Making Tax Digital (MTD) API Submission Flow Architecture

How the System Works: 1. Accountant reviews the finalized 9-box VAT figures and clicks 'Submit to HMRC MTD'. ➔ 2. Blynt validates the active HMRC OAuth2 cryptographic connection. ➔ 3. Constructs the official HMRC MTD JSON payload conforming to HMRC API specifications. ➔ 4. Transmits the payload securely over TLS to HMRC's government gateway. ➔ 5. HMRC validates the submission and returns an official cryptographic submission receipt reference.
Outcome: The business complies 100% with UK Making Tax Digital legal filing mandates. | **Guardrail:** If HMRC rejects a figure (e.g., invalid period key), Blynt displays the exact government gateway message.

UC-096: Viewer Role Write Operation Block Enforcement

Module: Access Control | **Actor:** External Auditor / Viewer | **Rejection:** 403 VIEWER_READ_ONLY

Business Goal: Allow auditors to inspect books without any risk of changing data.

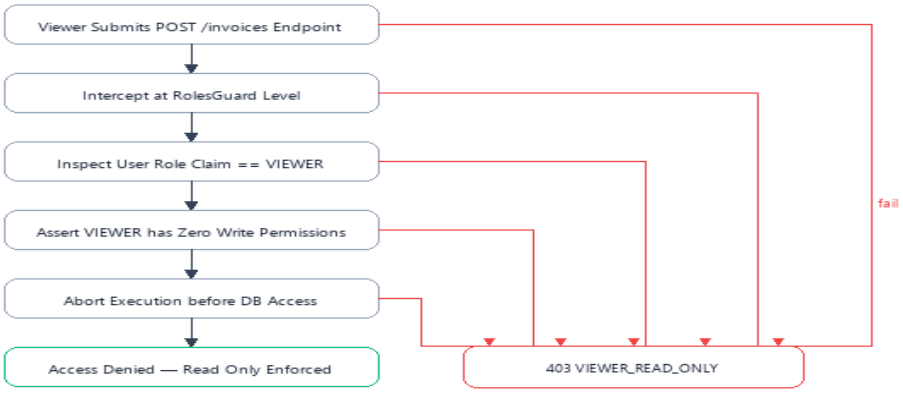


Figure 096 — Viewer Role Write Operation Block Enforcement Flow Architecture

How the System Works: 1. A user with the assigned role 'VIEWER' logs into the system. ➔ 2. The UI hides all 'Create', 'Edit', 'Delete', and 'Post' action buttons. ➔ 3. If the viewer attempts to send an HTTP POST request to create an invoice, the RolesGuard intercepts it. ➔ 4. The guard checks the user's role claim against endpoint metadata. ➔ 5. The request is rejected immediately with HTTP 403 Forbidden before reaching the database.
Outcome: Auditors and read-only directors can inspect accounts with complete safety. | **Guardrail:** The screen alerts: 'Your account has View-Only permissions. You cannot perform this action.'

UC-097: Bookkeeper Settings Modification Prohibition

Module: Access Control | **Actor:** Bookkeeper | **Rejection:** 403 BOOKKEEPER_RESTRICTED

Business Goal: Prevent operational staff from altering company bank accounts or tax settings.



Figure 097 — Bookkeeper Settings Modification Prohibition Flow Architecture

How the System Works: 1. A user with role 'BOOKKEEPER' attempts to change the company bank account details in Settings. ➔ 2. Blynt's PermissionGuard intercepts the request. ➔ 3. The guard asserts that modifying banking infrastructure requires role 'ADMIN' or 'OWNER'. ➔ 4. The modification is rejected. ➔ 5. The unauthorized attempt is recorded in the security audit log.
Outcome: Internal controls protect against unauthorized redirection of company funds. | **Guardrail:** The screen shows: 'Administrator permissions required to update banking settings.'

UC-098: Accountant Period Close Authority Validation

Module: Governance & Compliance | **Actor:** Bookkeeper vs Accountant | **Rejection:** 403 ACCOUNTANT_ROLE_REQUIRED

Business Goal: Ensure only qualified financial personnel can lock company accounts.



Figure 098 — Accountant Period Close Authority Validation Flow Architecture

How the System Works: 1. A junior bookkeeper clicks 'Lock Financial Year'. ➔ 2. Blynt checks the user's role in the organization token. ➔ 3. The system detects that the user is a 'BOOKKEEPER', not an 'ACCOUNTANT' or 'OWNER'. ➔ 4. The action is blocked. ➔ 5. A notification explains that only certified accountants or business owners can close financial years.
Outcome: High-impact statutory decisions are restricted to qualified decision-makers. | **Guardrail:** The screen displays: 'Only an Accountant or Owner can close financial periods.'

UC-099: Audit Trail Cryptographic SHA-256 Hash Chain

Module: GRC Compliance | **Actor:** Audit Service | **Rejection:** 500 AUDIT_CHAIN_BROKEN

Business Goal: Prove to tax inspectors that audit logs have never been altered.

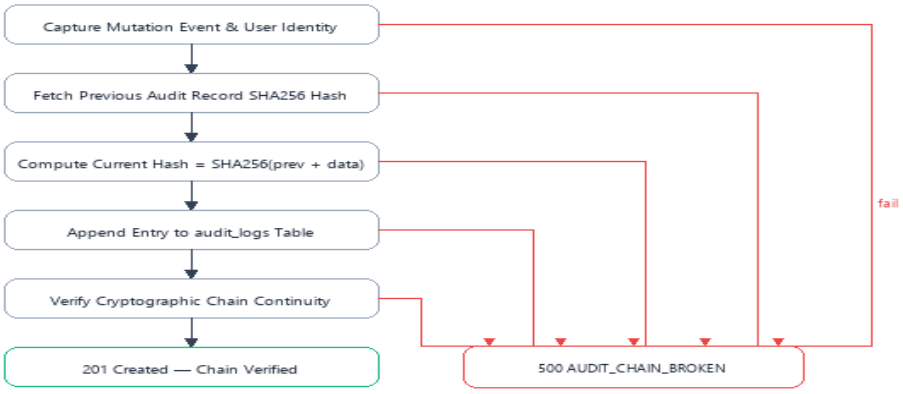


Figure 099 — Audit Trail Cryptographic SHA-256 Hash Chain Flow Architecture

How the System Works: 1. Whenever any financial record is created, edited, voided, or viewed, an audit event is triggered. ➔ 2. Blynt captures the timestamp, user ID, IP address, action, and payload diff. ➔ 3. The system fetches the SHA-256 hash of the immediately preceding audit record. ➔ 4. Computes a new hash: SHA-256(previousHash + timestamp + payload). ➔ 5. The event is appended to the audit table, creating a tamper-evident cryptographic blockchain.
Outcome: Provides statutory proof to HMRC that company accounting logs have not been manipulated off-the-books. | **Guardrail:** Any broken link in the hash chain immediately triggers an automated compliance alert.

UC-100: Database Direct Deletion Prevention Trigger

Module: Statutory Immutability | **Actor:** PostgreSQL Engine | **Rejection:** 500 DIRECT_SQL_MODIFICATION_BLOCKED

Business Goal: Block direct SQL attacks from deleting compliance audit trails.



Figure 100 — Database Direct Deletion Prevention Trigger Flow Architecture

How the System Works: 1. A malicious attacker or rogue database administrator runs: 'DELETE FROM audit_logs;'. ➔ 2. A PostgreSQL database-level 'BEFORE DELETE OR UPDATE' trigger fires instantly. ➔ 3. The trigger executes an unconditional exception: 'RAISE EXCEPTION "AUDIT LOGS ARE IMMUTABLE";'. ➔ 4. The database transaction is aborted and rolled back immediately. ➔ 5. Zero rows are deleted; 100% of the audit trail is permanently preserved.
Outcome: Compliance records are physically protected at the database engine level against any deletion. | **Guardrail:** Database returns: 'Audit logs are permanent and immutable. Direct deletion is prohibited.'